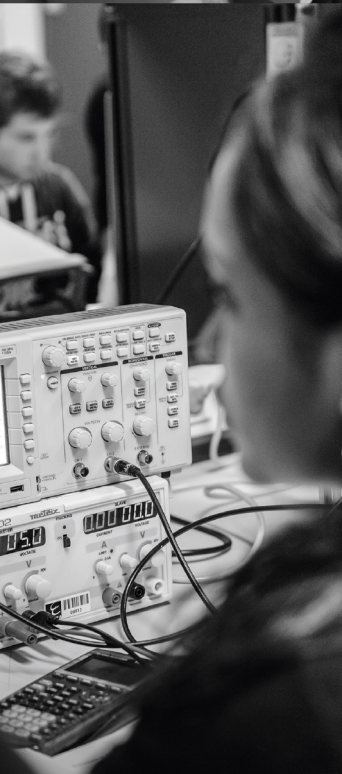




ENSEA

Beyond Engineering

SYSTÈMES À MICROCONTRÔLEUR

DTI_1504

Sujets de Travaux Dirigés et Travaux Pratiques

1^{re} année AB

Contents

I. Amphithéâtre	5
II. Texte des séances de TD (5 séances de 4 heures)	7
1. TD 1 - Représentation des nombres et découverte de la chaîne de compilation	8
1.1. Arithmétique binaire	8
1.1.1. Interprétation hexadécimale	8
1.2. Mise en place d'un projet vide	8
1.3. Exercice - découverte des fonctions HAL de gestion des LEDs	10
1.4. Exercice - usage du débogueur	10
1.5. Hello world !	11
1.6. Un vrai exercice de programmation	11
1.6.1. Fonction télérupteur - 1	11
1.6.2. Fonction télérupteur - 2	12
1.6.3. Fonction télérupteur - 3	12
1.7. Amélioration des codes précédents	12
2. TD 2 - Types, tableaux, pointeurs, portée et durée de vie des données en C	13
2.1. Quiz sur le TD1 (1 heure) - minuterie	13
2.2. Représenter des nombres	13
2.3. Dépassement des limites de calcul	14
2.4. Portée des variables	15
2.5. Pointeurs	16
2.6. Tableaux	16
2.7. Un petit exercice avec des tableaux	17
3. TD 3 - Parlons un peu algorithmique	18
3.1. Quiz sur le TD2 (1 heure) - Caractéristiques d'un tableau d'entiers	18
3.2. Tri par recherche de minimum	18
3.3. Tri par bulle	20
4. TD 4 - Structures	21
4.1. Quiz sur le TD3 (1 heure)	21
4.1.1. Tri par insertion	21
4.2. Un décodeur de code Morse	22
4.2.1. Projet de démarrage	22
4.2.2. Recherche dans l'alphabet	23
4.2.3. Captation de la durée d'appui	23
4.2.4. Décodage des appuis	23
4.2.5. Codage sur la LED	23
5. TD 5 - Liaisons et bus	24
5.1. Quiz sur le TD4 (1 heure 30)	24
5.2. Liaisons et bus	25
5.2.1. Liaison USART	25

5.2.2. Liaison I2C	26
5.3. Programmation interruptive	27
III. Texte de TP (5 fois 4 heures)	28
6. TP1 et 2 - Gestion de chaîne de caractères - Norme NMEA	29
6.1. Objectifs	29
6.2. Setup matériel du projet	29
6.3. Les liaisons séries	30
6.4. Setup logiciel du projet	31
6.5. Les trames NMEA	31
6.5.1. Décodage	31
6.5.2. Talker ID	31
6.5.3. Type de trames	32
6.5.4. Deux exemples de types de trames	32
6.6. Récupération des trames NMEA sur la liaison USART1	32
6.6.1. Vérification du paramétrage de la liaison série USART1	32
6.6.2. Boucle de recopie entrée / sortie	34
6.6.3. Fonction fillBuffer	34
6.7. Vérification de l'intégrité des trames	36
6.8. Récupération de l'information au format flottant	37
6.9. Affichage de l'information et test du module réel.	38
6.10. Recherche de la ville la plus proche	39
6.10.1. Création de la base de donnée	39
6.10.2. Calcul de distance	40
6.10.3. Recherche du minimum	41
7. TP3 - Un framework multithreadé minimaliste.	42
7.1. Introduction - préparation	42
7.1.1. Programme en polling	42
7.1.2. Programme en multi tâche simple	42
7.2. Mise en place du projet de base.	43
7.3. Le scheduler, usage du timer.	43
7.3.1. Calcul des paramètres du timer	43
7.4. La tâche de gestion des LEDs.	44
7.4.1. Le driver logiciel des LEDs.	44
7.4.2. La tâche taskLED().	46
7.5. La tâche de gestion des boutons.	47
7.5.1. La tâche taskButton().	47
7.6. La tâche de gestion de l'écran.	48
8. TP 4 - Des usages multiples des Timers	49
8.1. Préparation : A FAIRE AVANT LE TP !	49
8.1.1. Environnement matériel - moteur	49
8.1.2. Questions de préparation relatives au moteur pas à pas.	50
8.1.3. Environnement matériel - le codeur rotatif	51
8.2. Travail demandé - partie I : le driver du codeur incrémental	52
8.3. Travail demandé - partie II : le driver du moteur pas à pas	53
8.4. Travail demandé - partie III : mise en lien des deux parties	54
8.5. Travail optionnel : amélioration du driver de moteur pas à pas.	54
8.6. Annexe : une correction du driver	54

9. TP4 - Liaisons et bus, consommation électrique	57
9.1. Mesure de la consommation électrique.	57
9.1.1. Préparation	57
9.1.2. Manipulation de base	58
9.1.3. Mode power down	59
9.2. Liaisons et bus	60
9.2.1. Le composant BMM150	60
9.2.2. Détection de l'angle par rapport au Nord magnétique	61
9.3. Le projet complet (plus qu'optionnel)	61



Les TDs et TP4 sont réalisés avec vos PC personnels en utilisant Cube IDE, une suite gratuite de développement (en fait un *plugins* d'Eclipse), un IDE (*Integrated Debug Environment*) est un outil dont la maîtrise est à peu près indispensable à un informaticien.

Cube IDE reposant sur Java, il peut s'exécuter sur Windows, Linux et Mac OS. Vous **devez** l'avoir installé **avant** la première séance de TD.

La carte de développement stm32 nucleo L476rg a déjà été distribuée.

Ensemble de texte écrit à l'été par A. Tauvel. Un grand merci à tous les relecteurs et rédacteur de Quiz, notamment L. Monchal et N. Simond.

Part I.

Amphithéâtre

Les vidéos de cours sont disponibles à l'adresse : https://ensea.ubicast.tv/channels/#c_1a_base

Voici les thèmes des quatre séances d'amphithéâtre :

- Hardware
 - Périphérique élémentaire : les GPIOs
 - Le périphérique TIMER
 - * Mesure du temps
 - * Codeur rotatif
 - * PWM
 - Périphériques de communications
 - * USART
 - * Le Bus I2C
 - * Le Bus SPI
 - Modèle de consommation
- Software
 - Langage C : types
 - Langage C : fonction
 - Langage C : bit banging
 - Langage C : les types énumérés
 - Préprocesseur
 - Modèle mémoire
 - Tableaux
 - Pointeurs
 - Chaîne de caractères
 - Structure

Part II.

**Texte des séances de TD (5 séances de
4 heures)**

1. TD 1 - Représentation des nombres et découverte de la chaîne de compilation

Dans ce TD, après avoir revu un peu d'arithmétique binaire, nous allons apprendre à utiliser une chaîne de compilation et revoir les structures de base de la programmation appliquées ici au langage C :

- Test avec if
- Test avec switch case
- Boucle for
- Boucle while

Ces éléments sont des prérequis indispensables, qu'il faut savoir manipuler dans tous les langages.

1.1. Arithmétique binaire

1.1.1. Interprétation hexadécimale

Considérons le code suivant :

```
1 void fonction(void){
2     uint8_t variable_1 = 19;
3     uint8_t variable_2 = 147;
4     uint8_t variable_3 = 0xD1;
5     int8_t variable_4 = (int8_t) variable_3;
6     uint8_t variable_5 = variable_1 - 25;
7 }
```



- Quelles sont les valeurs binaires correspondantes aux différentes variables ?
- Comment représente-t-on ces valeurs en hexadécimal ?
- Quelle est la valeur décimale de variable_3 ?
- Quelle est la valeur décimale de variable_4 ?
- Quelle est la valeur décimale de variable_5 ?
- Quelle est la valeur décimale de variable_4 + 47 ?

1.2. Mise en place d'un projet vide

Téléchargez sur Moodle le fichier `onlyNucleoL476.ioc`. Ce fichier décrit les trois périphériques que nous allons utiliser : la liaison série, la LED et le bouton poussoir. À l'aide du tutoriel en ligne

sur Moodle. Réaliser un projet microcontrôleur avec STM32Cube, déployez le projet complet. Pour cela, voici les étapes principales :

- Allez dans le menu File / New / STM32 Project from an existing .ioc file.
- Générez votre code (cela nécessite un compte ST).
- Ajoutez le code nécessaire au fonctionnement de printf.
- Ajoutez l'appel à une fonction setup() avant la boucle infinie, et à une fonction loop() dans la boucle infinie.
- Ajoutez au projet un fichier utils.c et un fichier utils.h définissant ces deux fonctions.
- Testez la compilation, et en particulier la fonction printf.

```
1
2  #include <stm32l4xx_hal.h>
3  #include "utils.h"
4
5  void setup(){
6      // Insert the setup code here, it will run once.
7  }
8
9  void loop(){
10     // This code will run indefinitely.
11 }
```

Listing 1: Le code du fichier utils.c

```
1
2  void setup(void);
3  void loop(void);
4
```

Listing 2: Le code du fichier utils.h

Connectez votre carte au PC à l'aide du câble USB. En appuyant sur le symbole de l'insecte (voir figure suivante), passez en mode débogueur. Le logiciel vous propose alors de changer de perspective, c'est-à-dire de modifier la manière dont vous interagissez avec le projet.

La notion de perspective est importante dans Eclipse (STM32Cube est un plugin d'Eclipse). Vous en utiliserez principalement deux : celle permettant d'écrire du code et celle permettant de déboguer. Si vous ne trouvez pas une commande, il est possible que vous ne soyez pas dans la bonne perspective. L'apprentissage d'Eclipse présente une certaine difficulté au début : il est normal de s'y sentir un peu perdu. Accrochez-vous !

1.3. Exercice - découverte des fonctions HAL de gestion des LEDs

Insérez dans votre fonction loop les instructions suivantes :

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
HAL_Delay(250);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_Delay(250);
```

Listing 3: Notre premier programme !

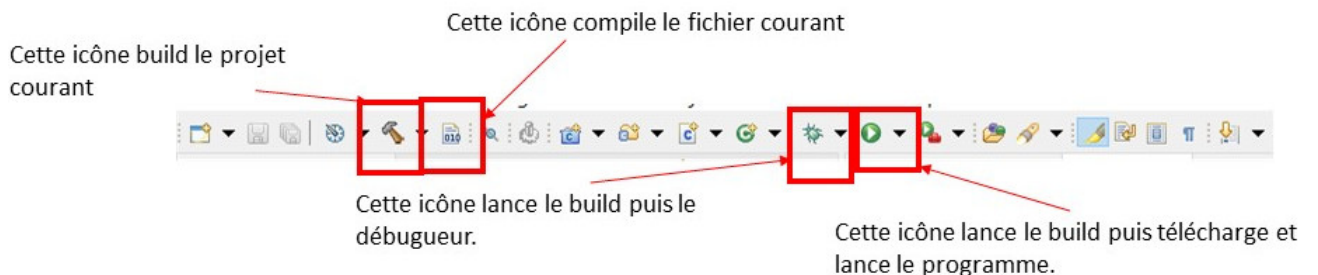


La LED est câblée physiquement sur la broche 5 du port A (PA5). Comment cette information apparaît-elle dans le code que vous venez de taper ? À l'aide d'un clic droit dans le code et de "Open Declaration", suivez la définition de la macro GPIO_A. Comment et où est-elle définie ? Ce type de définition repose sur le préprocesseur : elles ne correspondent donc pas à des instructions exécutées. Toutes les instructions préprocesseur commencent par le symbole #, comme ici #define. Ajoutez une macro définissant deux nouvelles étiquettes : GPIO_PORT_LED et GPIO_PIN_LED, renvoyant aux macros d'origine. Modifiez le code et vérifiez son bon fonctionnement.

1.4. Exercice - usage du débogueur

En appuyant sur l'icône en forme d'insecte, vous passez en mode débogueur. En double-cliquant à gauche d'une ligne, vous insérez un point d'arrêt. Trois modes de fonctionnement pas à pas sont disponibles :

- Step into
- Step over
- Step out



Dans le programme loop précédent, placez un point d'arrêt sur la première instruction. Passez en mode débogueur et lancez l'exécution par "Resume" (raccourci F8). Attendez à chaque fois que l'exécution se suspende avant de continuer. Testez les trois modes d'exécution pas à pas. Quelles différences observez-vous entre eux ?

1.5. Hello world !

La fonction `printf` permet d'envoyer une chaîne formatée de caractères ASCII. Par exemple, l'expression :

```
printf("ENSEA promotion %d, votre note %f \n\r", annee, note);
```

affiche la variable `annee` sous forme d'entier (`%d` = décimal) et la variable `note` sous forme de nombre flottant (`%f`).

Attention ! Les deux derniers caractères (`\r\n`) marquent la fin de ligne : **carriage return** et **line feed**, comme sur les machines à écrire. Si vous les oubliez, le processeur placera bien les données en mémoire mais ne les enverra pas immédiatement, attendant d'en accumuler davantage. Ces caractères sont donc obligatoires dans notre implantation du C (ce n'est pas toujours le cas ailleurs).

Pour voir le message, il faut utiliser une console. Vous pouvez employer un logiciel externe comme TeraTerm ou bien la console intégrée de STM32CubeIDE. Cette dernière est accessible via l'onglet Console en mode débogueur (voir vidéos de cours et démonstrations en amphi).

Les paramètres de la liaison série transitant via l'USB sont : 115 200 bits par seconde, sans bit de parité. (Pas d'inquiétude : vous comprendrez bientôt ce que cela signifie !)

Vous pouvez également consulter ce document : <https://moodle.ensea.fr/mod/resource/view.php?id=35626> pour plus d'informations.



Dans le programme précédent, ajoutez un message de bienvenue au démarrage. Testez votre programme.

Optionnel : scindez l'émission du message en deux instructions `printf`. À l'aide du mode pas à pas (Step over), vérifiez que le message est bien envoyé en entier, mais qu'il n'apparaît qu'après la seconde instruction.



Dans le programme précédent, ajoutez un message affichant le nombre de clignotements de la LED. Pour cela, vous aurez besoin d'une variable de type `int`. Cette variable doit-elle être locale (définie dans le corps de la fonction) ou globale (définie en dehors du corps de la fonction, mais dans le fichier) ? Argumentez votre réponse.

Ajoutez le code pour initialiser puis incrémenter la variable.

Testez votre programme.

1.6. Un vrai exercice de programmation

1.6.1. Fonction télérupteur - 1



À partir d'un projet vierge, écrivez un programme qui attend en boucle un appui sur l'interrupteur.

Deux indices : l'interrupteur est relié à la broche PC13. La fonction permettant de lire une broche est `HAL_GPIO_ReadPin()`. Vous devrez consulter la documentation.

Important : un appui correspond ici à la lecture de la valeur 0.

À chaque appui sur l'interrupteur, on change l'état d'une variable booléenne stockée dans un entier. Une astuce simple : `etat_led = 1 - etat_led;`

N'oubliez pas qu'il faut attendre le relâchement du bouton pour ne pas compter plusieurs fois le même appui.

1.6.2. Fonction télérupteur - 2



Modifiez le programme précédent de manière à ce que l'état de la LED soit affiché sur la console sous la forme d'un message "Switch pressed, LED ON" ou "Switch pressed, LED OFF".

1.6.3. Fonction télérupteur - 3



Modifiez le programme précédent afin de compter le nombre d'appuis, et affichez un message du type : "Switch pressed N times, LED state : 0/1".

1.7. Amélioration des codes précédents

La plupart d'entre vous ont probablement utilisé des boucles `while()` très longues et la fonction `HAL_Delay()`. Or cette fonction n'est pas interruptible : le programme reste bloqué pendant la durée du délai, ce qui empêche toute autre tâche parallèle.

Il est donc intéressant de réécrire le programme sans utiliser `HAL_Delay()`. Pour cela, il faut raisonner dans une boucle infinie. Par exemple :

```
À L'INFINI :  
    touche_actuelle = lecture_de_la_touche()  
    SI touche_actuelle ET pas memoire_touche ALORS  
        change_etat_led()  
    memoire_touche = touche_actuelle
```



Réécrivez le programme précédent sans utiliser de code bloquant, de manière à ce qu'il soit non-interruptible.

2. TD 2 - Types, tableaux, pointeurs, portée et durée de vie des données en C

2.1. Quiz sur le TD1 (1 heure) - minuterie



- Rechargez un projet vierge à l'aide du tutoriel en ligne.
- Le programme attend que l'interrupteur soit appuyé.
- Après chaque appui, la LED s'allume pendant une durée définie par une macro (par exemple 1500 millisecondes).
- À chaque appui, un message apparaît sur la console : "Light ON".
- À la fin de chaque temporisation, un message apparaît sur la console : "Light OFF".

2.2. Représenter des nombres

En Python, les variables ne sont pas typées de manière explicite : le langage utilise un typage dynamique, ce qui est flexible mais peut parfois induire des erreurs de compréhension. En C, les variables sont typées : il faut indiquer au compilateur comment manipuler chaque variable. (On dit que le C est faiblement typé, car il est possible de modifier le type d'une variable en cours d'utilisation.)

Trois types primitifs sont massivement utilisés :

- `char` : représente souvent des caractères, stockés sur 8 bits.
- `int` : représente des nombres entiers (positifs ou négatifs), stockés sur la taille naturelle du processeur (ici 32 bits).
- `float` : représente des nombres flottants avec mantisse et exposant (notation scientifique), généralement sur 32 bits.

Soit le code suivant :

```

1
2  #include <stm32l4xx_hal.h>
3  #include "ensea.h"
4
5  void setup(void){
6      int ceci_est_une_variable = 100;
7      printf("Variable selon différents formats : %d, %x, %c \n\r",
8             ceci_est_une_variable,
9             ceci_est_une_variable,
10            ceci_est_une_variable);
11
12     ceci_est_une_variable++;
13     printf("Après modification : %d, %x, %c \n\r",
14            ceci_est_une_variable,
15            ceci_est_une_variable,
16            ceci_est_une_variable);
17 }
18
19 void loop(void){
20     // This code will run infinitely.
21 }

```

Listing 4: Le type entier avec trois visualisations



Qu'affiche le programme ci-dessus ?

Interprétez ces résultats.

Que remarquez-vous concernant l'affichage du caractère ASCII (%c) ?

Remarque : les types primitifs du C ne sont pas toujours pratiques, car leurs signes (signés ou non signés) et leur taille exacte dépendent de la plateforme. On leur préfère des types comme `uint8_t` (entier non signé sur 8 bits), définis à l'aide de `typedef` dans des fichiers d'en-tête.

2.3. Dépassement des limites de calcul

Un entier non signé sur 8 bits prend des valeurs comprises entre 0 et 255.

Soit le code suivant :

```

1  uint8_t c;
2  for(c=0; c<=254; c++){
3      printf("c = %d %x %c \n\r", c, c, c);
4  }
5  printf("c = %d %x %c \n\r", c, c, c);
6  c++;
7  printf("c = %d %x %c \n\r", c, c, c);

```



Interprétez les affichages du code ci-dessus.

À retenir : Les dépassements de capacité (appelés *overflows*) sont des problèmes majeurs en informatique. À chaque calcul, il faut vérifier que le résultat reste dans les limites de la variable utilisée.

2.4. Portée des variables

Soit le code suivant :

```
#include <stm32l4xx_hal.h>
#include "ensea.h"

int globale = 100;

void fonction(int parametre){
    int locale = 5;
    locale++;
    globale++;
    parametre++;
    printf("*** Intérieur de la fonction *** \n\r Valeur de locale : %d
    \t de globale : %d \t de parametre : %d\n\r",
        locale, globale, parametre);
}

void setup(void){
    int locale = 10;
    int parametre = 20;
    printf("*** Setup avant appel de fonction ***\n\r
    Valeur de locale : %d
    \t de globale : %d \t de parametre : %d\n\r",
        locale, globale, parametre);
    fonction(parametre);
    printf("*** Setup après appel de fonction ***\n\r
    Valeur de locale : %d
    \t de globale : %d \t de parametre : %d\n\r",
        locale, globale, parametre);
}

void loop(void){
    // This code will run indefinitely.
}
```

Listing 5: Démonstration sur les portées de variables



Compilez et exécutez le programme ci-dessus.
Interprétez les trois affichages obtenus.
Notez soigneusement vos conclusions, elles sont importantes.

À retenir :

- Une variable locale n'existe qu'à l'intérieur de sa fonction. Deux variables locales de même nom dans deux fonctions différentes ne désignent pas la même donnée.
- Une variable globale est accessible depuis n'importe quelle fonction. Cela peut faciliter certaines choses, mais aussi introduire des erreurs.
- Lors du passage d'un paramètre à une fonction, une copie de la valeur est effectuée. Les modifications faites à l'intérieur de la fonction ne sont donc pas conservées.

2.5. Pointeurs

Soit le code suivant :

```
void stupid_function(int stupid_variable){
    stupid_variable++;
}

void another_function(void){
    int x = 0;
    stupid_function(x);
    printf("Valeur de x : %d", x);
}
```

Listing 6: Un exemple pour introduire les pointeurs



Qu'affiche ce programme et surtout... pourquoi ?
 Créez une fonction `void not_so_stupid_function(...)` qui fasse en sorte que `x` vaille 1 à la fin de l'exécution.



À retenir : Un pointeur passé en paramètre permet souvent de modifier une variable en sortie de fonction.
 C'est notamment le cas de la fonction `scanf`, qui lit une valeur entrée par l'utilisateur.

2.6. Tableaux

Soit le programme suivant :

```

#define ARRAY_SIZE 10

void setup(void){
    setvbuf(stdin, NULL, _IONBF, 0);
    // Ce setup est indispensable pour scanf.
}

void displayArray(int array[ARRAY_SIZE]){
    int i;
    for (i = 0; i < ARRAY_SIZE; i++){
        printf("array[%d] = %d\n\r", i, array[i]);
    }
}

void loop(void){
    int array[ARRAY_SIZE] = {0};
    int index;
    while (1){
        scanf("%d", &index);
        array[index] = 1;
        displayArray(array);
    }
}

```

Listing 7: Gestion d'un tableau



Testez le programme ci-dessus.
 Avec le débogueur, observez la variable array.
 Que valent les 10 premières cases du tableau ? Et la onzième ?
 Que se passe-t-il si l'utilisateur saisit un nombre hors limite ?
 À partir de quelle valeur le programme adopte-t-il un comportement inattendu ?
 Que se passe-t-il si l'utilisateur saisit -1 ?

2.7. Un petit exercice avec des tableaux

Cet exercice est plus difficile : vous aurez peut-être besoin de l'aide de l'enseignant, mais il correspond au niveau attendu à la fin de ce cours.



Dans un projet vierge, écrivez une fonction `void fillArray (int * array, int size);` qui remplit les `size` cases du tableau avec la durée des `size` derniers appuis sur le bouton bleu.
Indice : la fonction `int HAL_GetTick()` donne la valeur de l'horloge système en millisecondes (durée depuis le reset de la carte).
 Testez votre fonction à l'aide du débogueur.
 Écrivez ensuite une fonction `int getMeanValue(int * array, int size);` qui calcule la moyenne des durées d'appui et la renvoie.

3. TD 3 - Parlons un peu algorithmique

3.1. Quiz sur le TD2 (1 heure) - Caractéristiques d'un tableau d'entiers



L'objectif de ce Quiz est très simple sur sa forme mais l'idée est de vérifier que les compétences présentées lors des 2 premières séances sont désormais acquises :

1. extraire à l'aide de la fonction `void caracArray(int array[ARRAY_SIZE], int* pmin, int* pmax, float* pmoy)` les valeurs minimale, maximale et moyenne (tronquée à 2 décimales) d'un tableau d'entiers. Vous pourrez découpler cette fonction en 2 ou 3 sous-fonctions si cela vous paraît plus simple,
2. afficher d'abord sur le terminal puis sur le port série de votre carte de développement les valeurs numériques obtenues avec un minimum de mise en forme (pas 3 valeurs numériques séparées par un espace ...) grâce à la fonction `void dispCaracArray(int mini, int maxi, float moy)`.

Deux options s'offrent à vous pour initialiser le tableau d'entiers :

- vous êtes parvenus à terminer l'exercice 2.7 du TD2, vous pouvez l'utiliser tel quel pour calculer puis afficher ses caractéristiques,
- dans le cas contraire, vous initiez chacune des cases de votre tableau en appelant la fonction `rand()` qui renvoie une valeur pseudo-aléatoire positive de type `int` dans l'intervalle $[0; 2^{31} - 1]$. Cette fonction doit être ensemencée/initialisée avec une graine (entier positif) qui vous assure que la séquence pseudo-aléatoire générée par les appels successifs à `rand()` soit originale à chaque exécution du code. Vous devrez donc initialiser le process en appelant une unique fois la fonction `srand(HAL_GetTick());` issue de la librairie `<stdlib.h>`.

3.2. Tri par recherche de minimum

Les algorithmes de tri sont un classique de programmation. Très utiles, on les classe en fonction de leur efficacité algorithmique.

Pour en savoir plus sur les algorithmes de tri, je vous engage à consulter le site <https://interstices.info/les-algorithmes-de-tri/>, d'excellente qualité.

Pour tester nos algorithmes de tri, nous allons dans un premier temps utiliser un tableau rangé aléatoirement, puis nous utiliserons l'exercice 2.7 précédent pour avoir un tableau pseudo aléatoire.

Le plus instinctif pour trier consiste à rechercher le minimum d'un tableau, le recopier dans un autre tableau, d'enlever ce minimum du tableau à trier et de recommencer...



Soit un tableau de n cases.

Combien de comparaisons faut-il faire pour trouver le minimum de ce tableau ?

Combien d'opérations élémentaires sont nécessaire au total pour opérer ce tri ?



Ecrivez une fonction ayant le prototype `int getMinPosition(int * array, int * toBeIgnored, int size);`. Cette fonction détermine **la position** (et non pas la valeur) du minimum du tableau `array` de taille `size`. Les éléments du tableau `toBelgnored` ont comme valeur 0 ou 1. Quand il valent '1', on ne tient pas compte de cet indice pour le minimum.

```
#define ARRAY_SIZE 15
int arrayTest[ARRAY_SIZE]={0, 1, -3, 10, -1, 0, 0, 0, 0, 12, 1024, 10, 45, 6, 4};

void loop(void){
    int ignored[ARRAY_SIZE]={0};
    int index;
    index=getMinPosition(arrayTest, ignored, ARRAY_SIZE);
    printf("First min at : %d \n\r",index);
    ignored[index]=1;
    index=getMinPosition(arrayTest, ignored, ARRAY_SIZE);
    printf("Second min at : %d \n\r",index);
}
```

Listing 8: Recherche de minimum dans un tableau



Ecrivez une fonction ayant le prototype `int sortArray(int * toSort, int * sorted, int size);`. Cette fonction a les actions suivantes :

- Initialisation d'un tableau `int sortedPosition[ARRAY_SIZE]` à 0.
- Pour tous les indices du tableau :
- Recherche du minimum avec comme interdiction de comparaison `sortedPosition`.
- Ecrire dans `sortedPosition[minimum]` la valeur 1.
- Ecrire dans `sorted[indice]` la valeur `toSort[minimum]`.

Testez avec le code suivant.

```
#define ARRAY_SIZE 15
int arrayTest[ARRAY_SIZE]={0, 1, -3, 10, -1, 0, 0, 0, 0, 12, 1024, 10, 45, 6, 4};

void loop(void){
    int ignored[ARRAY_SIZE]={0};
    int sorted[ARRAY_SIZE]={0};
    long start=HAL_GetTick();
    sortArray(arrayTest, sorted, ARRAY_SIZE);
    long stop=HAL_GetTick();
    printf("Solved in %d milliseconds\n\r", stop-start);
}
```

Listing 9: Tri naïf d'un tableau

Si le tri fonctionne mais que le temps n'est pas assez grand, augmentez la taille du tableau.

3.3. Tri par bulle

Au sein de cette grande famille d'algorithmes, le tri par bulle a pour intérêt d'être relativement simple à implémenter. Vous trouverez ci-dessous le pseudo code issue de Wikipédia de ce tri.

```
tri_à_bulles(Tableau T)
    pour i allant de (taille de T)-1 à 1
        pour j allant de 0 à i-1
            si T[j+1] < T[j]
                (T[j+1], T[j]) ← (T[j], T[j+1])
```

Listing 10: Tri par bulle.



Programmez un algorithme de tri par bulle, et appliquez le au programme précédent. Mesurez la durée du temps d'exécution, et comparez-là au tri "naïf".

4. TD 4 - Structures

4.1. Quiz sur le TD3 (1 heure)

4.1.1. Tri par insertion

Le tri par insertion est une technique naturelle de tri. C'est celle que vous utilisez spontanément lorsque vous devez trier un jeu de cartes : on décale la carte à trier jusqu'à trouver la bonne position.

L'image ci-dessous, extraite de Wikipédia, illustre le tri du tableau {6, 5, 3, 1, 8, 7, 2, 4}.

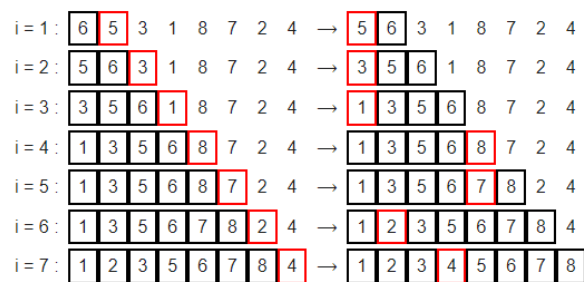


Figure 4.1.: Le tri par insertion en pratique

Le pseudo-code à mettre en œuvre est le suivant :

```
procédure tri_insertion(tableau T)
  pour i de 1 à taille(T) - 1
    # mémoriser T[i] dans x
    x ← T[i]

    # décaler les éléments T[0]..T[i-1] qui sont
    # plus grands que x, en partant de T[i-1]

    j ← i
    tant que j > 0 et T[j - 1] > x
      T[j] ← T[j - 1]
      j ← j - 1

    # placer x dans le "trou" laissé par le décalage
    T[j] ← x
```

Listing 11: Tri par insertion



Dans le projet du TD précédent :

- Codez la fonction de tri par insertion.
- Vérifiez son fonctionnement sur un tableau de petite taille.
- Testez le tri sur un tableau aléatoire de 1024 octets et comparez ses performances avec celles du tri naïf et du tri à bulles.

4.2. Un décodeur de code Morse

Nous voulons écrire un programme capable de décoder un message reçu en code Morse. Dans un premier temps, nous allons définir une structure de données permettant de représenter chaque lettre de l'alphabet.

```
typedef struct morse_lettre {
    char letter;
    char * code;
} MORSE_LETTER;

#define ALPHABET_SIZE 36

/*.....*/

const MORSE_LETTER alphabet[ALPHABET_SIZE] =
{
    {'A', "-.-"}, {'B', "-..."}, {'C', "-.-."},
    {'D', "-.."}, {'E', "."}, {'F', "..-."},
    {'G', "--."}, {'H', "...."}, {'I', ".."},
    {'J', "-.-.-"}, {'K', "-.-"}, {'L', "-.."},
    {'M', "--"}, {'N', "-."}, {'O', "---"},
    {'P', "-.-."}, {'Q', "--.-"}, {'R', "-.-"},
    {'S', "..."}, {'T', "-"}, {'U', "..-"},
    {'V', "...-"}, {'W', "-.-"}, {'X', "-.-.-"},
    {'Y', "-.-.-"}, {'Z', "--.."},
    {'1', "-.----"}, {'2', "-..----"}, {'3', "...----"},
    {'4', "....-"}, {'5', "....."}, {'6', "-...."},
    {'7', "--..."}, {'8', "---.."}, {'9', "----."},
    {'0', "-----"}
};
```

Listing 12: La structure morse_lettre et l'alphabet complet

4.2.1. Projet de démarrage



Créez un projet vierge comme indiqué dans le tutoriel. Intégrez la définition de la structure dans un fichier morse.h et celle de l'alphabet dans un fichier morse.c. Le programme utilisera la fonction printf, redirigée vers la console.

4.2.2. Recherche dans l'alphabet



- Écrivez la fonction `char lookupInBaseFromCode(char * code);` qui utilise `strcmp` (dans `string.h`) pour retrouver une lettre à partir de son code. Si la lettre n'est pas trouvée, la fonction renvoie le caractère `'0'`.
- Écrivez la fonction inverse : `char * lookupInBaseFromLetter(char letter);` qui renvoie le code associé à une lettre.
- Testez ces deux fonctions par des appels dans `main`.

4.2.3. Captation de la durée d'appui

Nous devons mesurer la durée d'appui sur le bouton. Trois cas sont possibles :

Définition	Symbole	Caractère
Appui court (inférieur à 200 ms)	court	'.'
Appui long (supérieur à 200 ms)	long	'-'
Absence d'appui pendant au moins une seconde	espace	' '

Pour interpréter correctement les appuis, il faut tenir compte de l'appui précédent. Par exemple, une absence d'appui de deux secondes ne doit pas générer deux espaces successifs, mais uniquement un seul.



- Écrivez une fonction `char waitForInput(char previousInput);` : si l'appui précédent est un espace, la fonction attend un nouvel appui et mesure sa durée ; sinon, après une seconde sans appui, elle renvoie un espace.
- Testez en affichant dans la console la séquence Morse saisie sur le bouton (par exemple : "... --- ...").

4.2.4. Décodage des appuis



Créez un tableau local de caractères pour enregistrer les entrées Morse entre deux espaces. Implémentez ensuite l'appel à la fonction de décodage afin d'afficher les caractères lisibles.

4.2.5. Codage sur la LED



Implémentez maintenant l'affichage en Morse sur la LED, à partir des caractères ASCII saisis dans la console.

5. TD 5 - Liaisons et bus

5.1. Quiz sur le TD4 (1 heure 30)



Il existe deux versions de cet exercice, en fonction de votre niveau :

- **Maître Jedi** : vous lisez l'énoncé ci-dessous, et essayez de le faire. Il n'est pas si dur que cela mais un peu long.
- **Padawan** : vous lisez l'énoncé ci-dessous, récupérez sur moodle à l'adresse <https://moodle.ensea.fr/mod/folder/view.php?id=7802> les sources d'un état avancé de l'exercice. Vous n'avez plus qu'à compléter les fonction `read_sensor` et `check_threshold`

Nous voulons ici définir deux structures permettant de simuler un réseau de capteur. Chaque capteur est modélisé par une structure contenant :

- Un entier nommé `id` qui est l'identifiant du capteur.
- Un flottant nommé `threshold` qui contient le seuil d'activation d'une alerte.
- Un flottant nommée `value` qui contient la valeur du capteur.
- Un énuméré nommée `state` de type `SENSOR_STATE` prenant comme valeurs possibles `ACTIVE` ou `INACTIVE`.



- Créez un projet vierge avec redirection du `printf` selon la méthode habituelle. Vous avez besoin de valider l'affichage des flottants via la fonction `printf`.
- Dans un fichier "`sensor.h`", définissez le type énuméré `SENSOR_STATE` et une macro `SENSOR_NUMBER` valant 5.
- Dans ce même fichier, créez une structure `SENSOR` définissant les 4 valeurs qui définisse l'état d'un capteur.
- Dans ce même fichier, créez une structure `SENSOR_ARRAY` comprenant un tableau de `SENSOR_NUMBER` variables de types `SENSOR`.

La fonction `void init_system(SENSOR_ARRAY * system);` met toutes les valeurs de chacun des capteurs à 0 ou `INACTIVE`. Cette fonction calcul un nombre aléatoire entre 0 et 10 par pas de 0.1 (`rand()%100/10`) pour le seuil de chaque capteur. Comme toujours, on pourra vouloir initialiser le générateur aléatoire avec la fonction `srand()`.

La fonction `void display_system(SENSOR_ARRAY * system);` affiche dans la console le détail de tous les capteurs les uns à la suite les autres. Pour afficher l'état, vous pouvez utiliser une expression ternaire élégante.

La fonction `void read_sensors(SENSOR_ARRAY * system)`; simule la lecture des capteurs en mettant pour chaque valeur une valeur aléatoire calculée entre 0 et 10 par pas de 0.1.

La fonction `void check_threshold(SENSOR * sensor)`; met à jour l'attribut `state` (ACTIVE ou INACTIVE) en fonction de la valeur `value` et du seuil `threshold` (au dessus du seuil, le capteur est ACTIVE).



- Codez la fonction `init_system` dans un fichier `sensor.c`
- Dans une fonction de test, implémentez un `SENSOR_ARRAY` nommé `system`. Appelez la fonction `init_system`.
- Testez à l'aide du debugger.
- Codez la fonction `display_system` dans ce même fichier.
- Testez l'affichage après initialisation.
- Codez la fonction `read_sensor`, et testez la en affichant le système après l'initialisation puis après une lecture des capteurs.
- Codez la fonction `check_threshold` et testez la en affichant le système après son appel.

5.2. Liaisons et bus

5.2.1. Liaison USART



- Si l'horloge du récepteur est décalée de $x\%$ par rapport à celle de l'émetteur, quelle doit être la valeur de x pour être certain d'échantillonner correctement tous les bits d'un octet, sachant que pour émettre un octet il faut émettre dix bits (start et stop inclus).
- En utilisant la formule de Bernoulli, si on note p la probabilité pour un bit d'être faux à la réception (canal symétrique : on a la même probabilité d'interpréter un 1 comme un 0 que l'inverse), quelle est la probabilité lorsqu'on utilise un bit de parité d'avoir une erreur de réception et de ne pas la détecter ? Faites l'application numérique pour $p = 10^{-6}$.

Soit la capture oscilloscope ci-dessous.

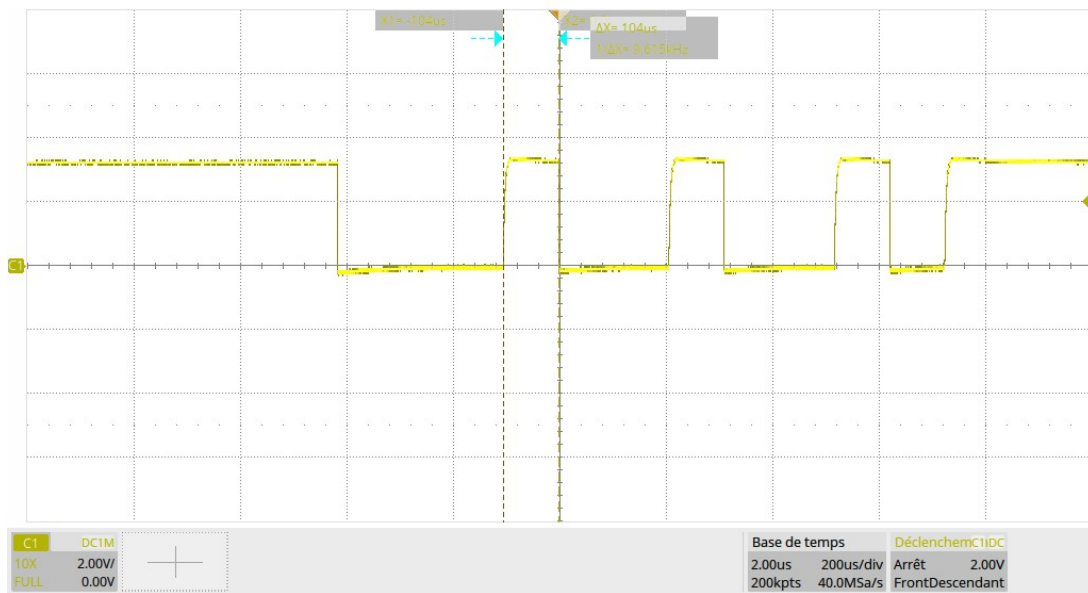


Figure 5.1.: Capture d'une liaison UART



La liaison étant au repos au début de la capture, répondez aux questions suivantes :

- Quelle est manifestement la vitesse de cette liaison ?
- Quel octet a été envoyé (il n'y a pas de contrôle de sécurité) ? Interprétez-le en code ASCII.

5.2.2. Liaison I2C

Soit la capture oscilloscope ci-dessous.

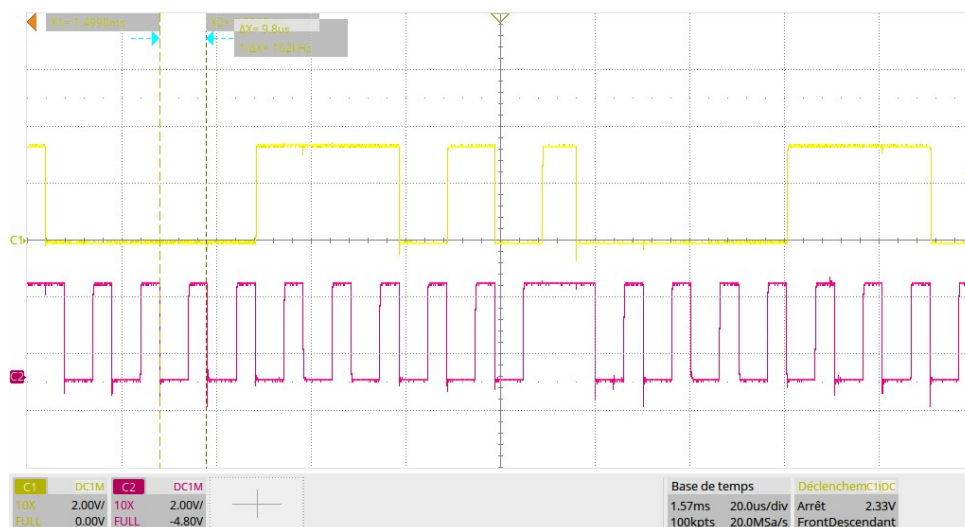


Figure 5.2.: Transfert IIC.

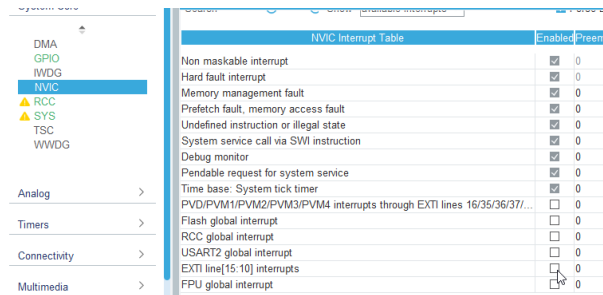


Figure 5.3.: Autorisation des interruptions externes



- Identifiez les voies de SDA et de SCL.
- Quelle est la vitesse de cette liaison ?
- Sur la capture, identifiez les conditions de START et de STOP, le bit d'acquittement (ACK).
- A quelle adresse accede-t-on ?
- Est-ce une lecture ou une écriture ?

5.3. Programmation interruptive

Sur votre carte, la patte PC_13 est reliée au contrôleur d'interruption.

Vous allez devoir ouvrir le fichier .ioc et activer la ligne d'interruption 13 (EXTI lines [15:10]) comme indiqué ci-dessous.

La fonction de callback doit s'écrire ainsi :

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == GPIO_PIN_13) {
        // Put your code here
    }
}
```



- Activez l'interruption de la ligne PC_13
- A l'aide d'un printf, vérifier le fonctionnement de votre interruption.
- Que pensez vous de l'usage d'un printf ici ?
- Communiquez à l'aide d'une variable globale pour que le printf se situe dans le main.
- Endormez le processeur entre deux interruptions.

Part III.

Texte de TP (5 fois 4 heures)

6. TP1 et 2 - Gestion de chaîne de caractères - Norme NMEA

6.1. Objectifs

Le but pédagogique de ce TP est d'apprendre à manipuler des chaînes de caractères en C via des pointeurs. Pour illustrer cela, nous allons manipuler des chaînes NMEA émises par un module GPS.

La norme NMEA 0183 (National Marine Electronics Association) définit la nature des messages envoyés entre équipements de type GPS. Cette norme ne présuppose pas de couche physique spécifique. Le module Grove-GPS Air employé dans ce TP émet ces trames en liaison série USART à la vitesse de 9600 bits par seconde. Comme la réception GPS indoor est de mauvaise qualité, les premières manipulations se feront à l'aide d'un simulateur écrit par votre serviteur et disponible en ligne via un lien github sur moodle.

Les différentes opérations de ce TP sont les suivantes :

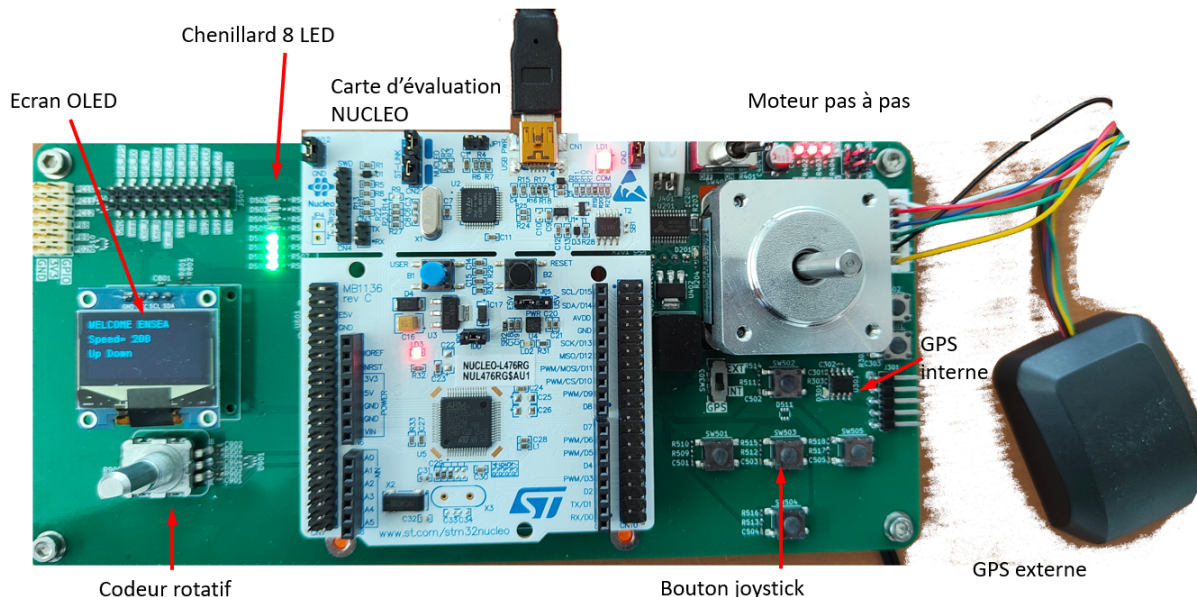
- Visualisation des trames NMEA à l'oscilloscope.
- Récupération des trames entrantes, être capable de détecter leur début et leur fin.
- Vérification de l'intégrité des trames reçus.
- Extraction des informations d'heure et de position.
- Transformation des informations d'heure et de position.
- Optionnel : affichage les informations d'heure et de position sur un écran OLED.

6.2. Setup matériel du projet

Nous¹ avons conçu une carte vous permettant de réaliser ce TP. Nous allons lors de ces deux TPs utiliser trois fonctionnalités de la carte :

- La liaison série USART2 pour le dialogue avec le PC.
- La liaison série USART1 pour le dialogue avec le vrai récepteur GPS et pour le dialogue avec le récepteur GPS émulé.
- De manière plus anecdotique, en fin de TP, l'écran OLED pour afficher latitude, longitude, et ville.

¹En l'occurrence, MM. Papazoglou et Tauvel



L'interrupteur situé en dessous du moteur permet de choisir la source des informations GPS sur la liaison USART1, entre GPS interne et GPS externe.

La LED rouge à proximité du processeur émulateur interne de GPS doit clignoter lorsque la carte est sous tension : cela indique une bonne émission des trames. Si ce n'est pas le cas, merci de ramener la carte au technicien.

Les deux boutons à droite du processeur émulateur interne de GPS permettent pour l'un de faire un reset, pour l'autre de changer de ville de référence.

6.3. Les liaisons séries



Le connecteur mâle noir de type "pinheader" au dessus de l'écran permet de visualiser les principaux signaux de la carte.

Branchez sur le signal GPS_RX une sonde d'oscilloscope (en n'oubliant pas la masse), visualisez les trames en trigger "NORMAL" et répondez aux questions suivantes :

- Que vaut l'état de repos de la ligne ?
- A quelle vitesse sont émises les données brutes (durée d'un bit) ?
- Le premier bit, dit "bit de start", est toujours un 0, ce qui permet de synchroniser la réception. Pourquoi et comment ?
- En tenant compte des bits de start et de stop, combien de bits envoie-t-on pour émettre un octet ?
- Quel est le débit binaire réel de la liaison ?
- En vous souvenant que l'envoi est "LSB first", décidez le premier octet envoyé. A quel caractère correspond-t-il ?

6.4. Setup logiciel du projet



Téléchargez en ligne le fichier .ioc disponible dans l'espace moodle du cours décrivant l'état des périphériques pour ces manipulations.

Générez le code de base du projet. Attention, pour cela il faut être connecté au serveur "ST" (Menu myST).

Rajoutez le code permettant l'accès à la fonction printf, disponible en ligne sur Moodle (fonction `int __io_putchar(int ch);`).

Rajoutez un code printf dans le programme principal et testez le fonctionnement global du projet.

6.5. Les trames NMEA

Le texte suivant est extrait de l'encyclopédie en ligne wikipédia :

Les trames NMEA sont codées au format ASCII et sont de la forme :

```
$<talker ID><Trame type>[,<données>,<données>]*<checksum>
```

6.5.1. Décodage

Champs	Longueur	Signification
\$	1	Marqueur de début de trame
Talker ID	2	Équipement ayant généré la trame NMEA
Trame type	3	Code identifiant le contenu de la trame
Données variable		Charge utile dont le contenu est défini par le "Trame type". Chaque valeur est séparée par le caractère ','
*	1	Séparateur de checksum
Checksum	2	Somme de contrôle générée par un ou exclusif de tous les caractères situés entre '\$' et '*' (exclus)
Fin de ligne	2	Caractères "carriage return" + "line feed" marquant un retour à la ligne

Exemple :

```
$GPGGA,064036.289,4836.5375,N,00740.9373,E,1,04,3.2,200.2,M,,.0,000*0E
```

6.5.2. Talker ID

Le type d'équipement à l'origine du signal (talker id) est défini par les deux caractères qui suivent le \$. Les principaux préfixes sont :

- BD ou GB - Beidou ;
- GA - Galileo ;
- GP - GPS ;
- GL - GLONASS.
- Le préfixe GN est utilisé dans le cas de signaux mixés GPS + GLONASS.

6.5.3. Type de trames

La longueur maximale d'une trame est 82 octets (en incluant les caractères de fin de ligne). Il existe plus d'une trentaine de trames NMEA différentes.

[...]

6.5.4. Deux exemples de types de trames

La trame GGA (Global Positioning System Fix Data) Elle est très courante car elle fait partie de celles qui sont utilisées pour connaître la position courante du récepteur GPS.

```
$GPGGA,064036.289,4836.5375,N,00740.9373,E,1,04,3.2,200.2,M,.,.,0000*0E
```

\$GPGGA : Type de trame

064036.289 : Trame envoyée à 06 h 40 min 36 s 289 (heure UTC)

4836.5375,N : Latitude 48,608958° Nord = 48° 36' 32.25" Nord

00740.9373,E : Longitude 7,682288° Est = 7° 40' 56.238" Est

1 : Type de positionnement (le 1 est un positionnement GPS)

04 : Nombre de satellites utilisés pour calculer les coordonnées

3.2 : Précision horizontale ou HDOP (Horizontal dilution of precision)

200.2,M : Altitude 200,2, en mètres

.,.,.,0000 : D'autres informations peuvent être inscrites dans ces champs

* : séparateur de checksum

0E : Somme de contrôle de parité, un simple XOR sur les caractères entre \$ et *

La trame RMC (Recommended Minimum Navigation Information)

Une autre trame très courante pour les bateaux est la RMC, qui donne l'heure, la latitude, la longitude, la date, ainsi que la vitesse et la route sur le fond mais pas l'altitude.

```
$GPRMC,053740.000,A,2503.6319,N,12136.0099,E,2.69,79.65,100106,.,.,A*53
```

\$GPRMC : type de trame

053740.000 : heure UTC exprimée en hhmmss.sss : 5 h 37 min 40 s

A : état A=données valides, V=données invalides

2503.6319 : Latitude exprimée en ddmm.mmmmm: 25° 03.6319' = 25° 03' 37,914"

N : indicateur de latitude N=nord, S=sud

12136.0099 : Longitude exprimée en dddmm.mmmmm: 121° 36.0099' = 121° 36' 00,594"

E : indicateur de longitude E=est, W=ouest

2.69 : vitesse sur le fond en nœuds (2,69 nd = 3,10 mph = 4,98 km/h)

79.65 : route sur le fond en degrés

100106 : date exprimée en jjmmaa : 10 janvier 2006

, : déclinaison magnétique en degrés (souvent vide pour un GPS)

, : sens de la déclinaison E=est, W=ouest (souvent vide pour un GPS)

A : mode de positionnement A=autonome, D=DGPS, E=DR

* : séparateur de checksum

53 : somme de contrôle de parité au format hexadécimal

6.6. Récupération des trames NMEA sur la liaison USART1

6.6.1. Vérification du paramétrage de la liaison série USART1

Vérifiez dans le fichier .ioc les caractéristiques de la liaison USART1. Elle doit fonctionner à 9600 bits par secondes.

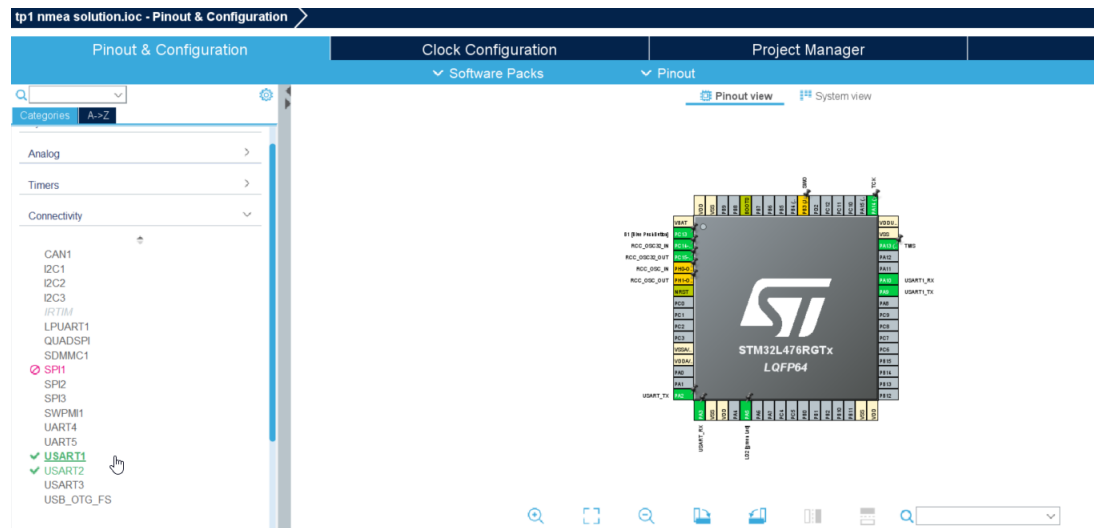


Figure 6.1.: Capture d'écran du générateur de code .ioc

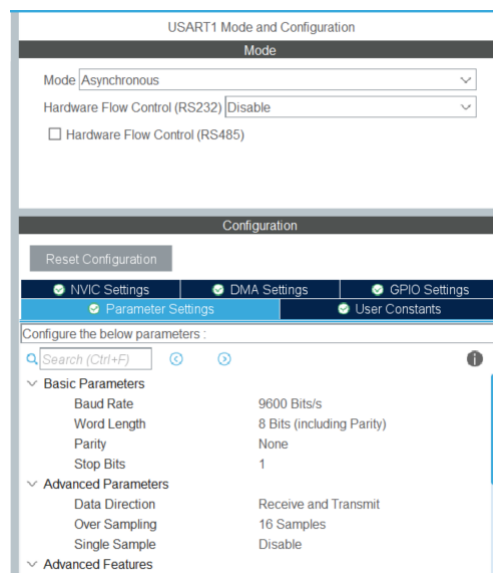


Figure 6.2.: Configuration de l'usart 1



- Vérifiez la configuration de la liaison USART1 comme indiqué ci-dessus.
- En cas de problème, régénérez le code en cliquant sur le bouton figurant un engrenage, ou bien cliquant CTRL+S.
- Suivez le tutoriel en ligne sur moodle pour ajoutez les fichiers "utils.h" et "utils.c" et obtenir un projet fonctionnel avec printf sur console et écran OLED.
- Ajouter à votre fichier de travail "utils.h" la directive `extern UART_HandleTypeDef huart1;`

6.6.2. Boucle de recopie entrée / sortie

Soit le code de "loop" suivant :

```
#include "main.h"

extern UART_HandleTypeDef huart1;
extern UART_HandleTypeDef huart2;

void loop(){
    char received_char;
    while(HAL_UART_Receive(&huart1, (uint8_t *)&received_char, 1, 0)!=HAL_OK){}
    HAL_UART_Transmit( &huart2, (uint8_t *) &received_char, 1, HAL_MAX_DELAY);
}
```

Listing 13: Code d'une boucle de réception / renvoi de caractères.



Testez le code ci-dessus.

6.6.3. Fonction fillBuffer

Selon les informations vues plus haut, on a une réception de 82 octets au maximum. On cherche à fabriquer une fonction au prototype suivant : `int fillBuffer(char * buffer, int size);` dans laquelle `buffer` désigne un pointeur sur un tableau de caractères, sortie de la fonction, et `size` un entier donnant la taille maximum du buffer. Cette fonction renvoie la taille de la trame reçue et remplit le tableau `buffer` avec les caractères reçus.

La fonction utilise une variable locale `received_char` qui contient le dernier octet reçu.

L'algorithme de la fonction est le suivant :

```
FONCTION fillBuffer :
variable locales : i, received_char, total;

    total <- 0;

    POUR index variant de 0 à size :
        buffer[index]=0;

    Annulation de la réception en cours.
        // Cette ligne permet de démarrer une réception proprement même si
        // elle démarre au milieu d'une trame.

    FAIRE
        received_char <- RECEPTION D'UN CARACTERE;
    TANT QUE received_char DIFFERENT DE '$'

    FAIRE
        VARIABLE POINTEE PAR buffer <- received_char ;
        received_char <- RECEPTION D'UN CARACTERE ;
        total <- total+1
```

```

        buffer <- buffer+1
TANT QUE received_char DIFFERENT DE '\n'

VARIABLE POINTE PAR buffer <- received_char ;

RENVOIE total
FIN

```

Remarque : la première boucle utilise l'adresse buffer comme un tableau, la troisième boucle comme un pointeur. Toutes les combinaisons sont possibles car un tableau correspond à un usage particulier d'un pointeur. Nous vous demandons cette forme particulière uniquement pour des raisons de pédagogie.



Codez la fonction fillBuffer en langage C dans un fichier "utils.c". Vous n'oublierez pas de la prototyper dans "utils.h".

Vous aurez pour cela besoin des fonctions HAL_StatusTypeDef HAL_UART_Abort(UART_HandleTypeDef *huart) et HAL_StatusTypeDef HAL_UART_Receive (UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout). Vous trouverez en ligne la documentation de ces fonctions et vous avez un exemple de fonctionnement dans la section précédente pour la deuxième.

A l'aide du fichier de test ci-dessous, testez le bon fonctionnement de votre programme. Vous pouvez aussi utiliser le debugger pour visualiser l'évolution de la variable "buffer".

Question : au sein de la fonction "fillBuffer", on incrémente la variable "buffer", et elle n'est jamais remise à sa valeur initiale. Pourquoi cela fonctionne-t-il normalement dans cette exécution en boucle de la fonction ?

```

#include <stdio.h>
#include "utils.h"

#define BUFFER_MAX_SIZE 128

void setup(){}

void loop(){

    char buffer[BUFFER_MAX_SIZE];
    int size;
    while(1){
        size=fillBuffer(buffer,BUFFER_MAX_SIZE);
        printf("Received %d char : %s \n\r",size,buffer);
    }
}

```

Listing 14: Code de test de la fonction "fillBuffer"

6.7. Vérification de l'intégrité des trames

Une trame est valide à 2 conditions :

- Elle commence par les lettres "\$GPGBA"
- Un OU EXCLUSIF de tout les caractères entre \$ et * donne les deux caractères finaux (en excluant les deux caractères \$ et * du calcul).

Par exemple : la chaîne "ABd" se code 0x41 0x42 0x64. Un ou exclusif bit à bit de ces valeurs donne le résultat : 0x67. Les deux derniers caractères doivent donc être '6' (0x36) et '7' (0x37).

Il nous faut donc être capable d'extraire le checksum reçu dans la trame d'une part, et de calculer le checksum de la trame d'autre part. Cela se fait avec ces deux fonctions :

```
1  FONCTION extractChecksum :
2      TANT QUE VARIABLE POINTE PAR buffer DIFFERENT de '*' FAIRE
3          buffer <- buffer+1
4      FIN TANT QUE.
5
6      RENVOIE (buffer[1]&0x0f)*16 +(buffer[2]&0x0f);
7  FIN extractChecksum
8
9  FONCTION calculateChecksum :
10     variable checksum
11     checksum <- 0
12     buffer <- buffer+1
13     TANT QUE VARIABLE POINTE PAR buffer DIFFERENT de '*' FAIRE
14         checksum <- checksum ^ (VARIABLE POINTE PAR buffer)
15         buffer <- buffer+1
16     FIN TANT QUE.
17
18     RENVOI checksum;
19 FIN calculateChecksum
```



- Codez la fonction `int isGPGBA(char * frame);` qui renvoie '1' si la trame débute par les caractères \$GPGBA et -1 si ce n'est pas le cas.
- Codez les fonctions `int extractChecksum(char * buffer);` et `int calculateChecksum(char * buffer)` en langage C dans un fichier "utils.c". Vous n'oublierez pas de la prototyper dans "utils.h".
- Explicitez le calcul de la ligne 6.
- A l'aide du fichier de test ci-dessous, testez le bon fonctionnement de votre programme. Vous pouvez aussi utiliser le debugger. Corrigez les éventuelles erreurs.

Petite astuce : Vous pouvez commenter ou décommenter tout un bloc de code. Il suffit de le sélectionner et sur un clic droit d'aller dans le menu contextuel sur "source" puis sur "comment" ou "uncomment".

```

#include <stdio.h>
#include "utils.h"

#define BUFFER_MAX_SIZE 128
char buffer[BUFFER_MAX_SIZE];

void loop(){
    int size = fillBuffer(buffer, BUFFER_MAX_SIZE);
    printf("Just received %d bytes \r\n",size);
    if (isGPGBGA(buffer)==1){
        printf("Trame GPGBGA \r\n");
        printf("Checksum -> calculated %x \t extract %x \r\n",
            calculateChecksum(buffer), extractChecksum(buffer));
    }
}

```

Listing 15: Code de test des fonctions "calculateChecksum" et "extractChecksum"



Enfin, à l'aide des trois fonctions précédentes, codez la fonction `int checkFrame(char * buffer)`; qui renvoie 0 si la trame est fautive (checksum ou entête incorrect) et 1 si la trame est correcte. A vous de compléter le code de test (1 trame sur 3 est correcte dans notre exemple).

6.8. Récupération de l'information au format flottant

Notre trame est maintenant prête à être analysée. L'information utile concernant la latitude se trouve à partir de la deuxième virgule. Elle aura par exemple la forme : 4836.537,N (c'est à dire 48 degré, 36.537 minutes).

Pour l'extraire, nous allons procéder en deux temps :

- Nous allons faire un calcul pour convertir la valeur absolue donnée en ASCII. On peut transformer les chiffres ASCII (de 0x30 à 0x39) en leur équivalent décimal à l'aide d'un masque en ET avec la valeur 0x0f.
- Nous allons ensuite rechercher la virgule suivante. Derrière, on trouvera soit 'N' soit 'S' qui changera le signe du résultat.

Prenons l'exemple ci-dessus :

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]	x[8]	x[9]
'4'	'8'	'3'	'6'	'.'	'5'	'3'	'7'	','	'N'
0x34	0x38	0x33	0x36	0x2E	0x35	0x33	0x37	0x2C	0x4E

Le calcul fait est donc le suivant (pour alléger cette formule, $x[n]$ représente la valeur `f[n]&0xf`) :

$$latitude = (-1)^{N/S} \cdot \left[10x[0] + x[1] + \frac{1}{60} \times (10x[2] + x[3] + 0.1x[5] + 0.01 * x[6] + 10^{-2}x[7]) \right]$$



Codez la fonction `float getLatitude(char * frame);`. Cette fonction :

- Recherche la deuxième virgule de la trame (une sous-fonction qui s'exécute deux fois sera appréciée).
- Réalisez le calcul ci-dessus de la valeur absolue. Conseil : faites des variables intermédiaires de calcul pour éviter la noyade.
- Déterminez la position de la virgule suivante.
- Un test est bien plus simple que l'écriture $(-1)^S$ pour l'obtention du signe de la latitude. C'est l'occasion de réviser l'opérateur ternaire. (Une latitude "SUD" est une latitude négative, une latitude "NORD" est positive).



Sur le modèle de la fonction précédente, codez la fonction `float getLongitude(char * frame);` cette fonction :

- Recherche la quatrième virgule de la trame.
- Réalise un calcul similaire (vérifiez le nombre de chiffre sur la trame reçu, ce n'est pas le même que pour la latitude).
- Testez le à l'aide du debugger.

6.9. Affichage de l'information et test du module réel.

Téléchargez sur Moodle les bibliothèques de contrôle de l'écran OLED, puis utilisez-les pour afficher la position. Attention, pour que l'usage de `"%f"` avec la fonction `printf`, il est nécessaire de cocher une case spécifique dans l'IDE (clique droit sur le projet / propriétés / C/C++ Build / Settings / MCU Settings, puis cochez Use float with printf), comme vous le voyez sur la capture ci-dessous.

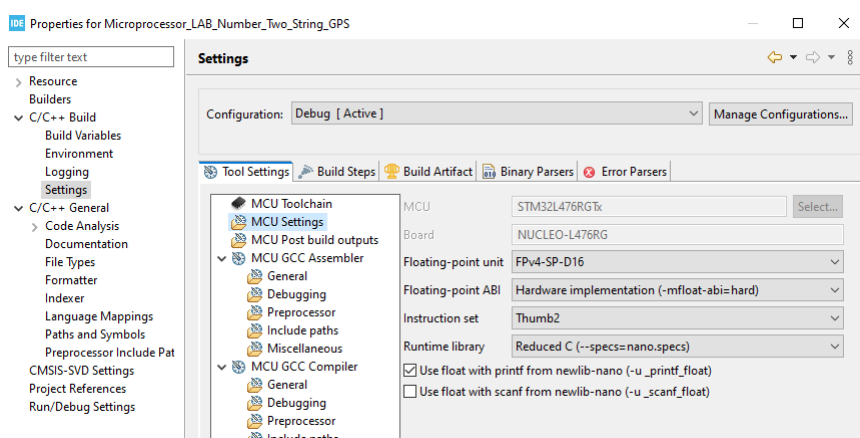


Figure 6.3.: Validation de l'affichage d'un flottant avec `printf`

Positionnez l'interrupteur GPS sur "EXT", et vérifiez le bon fonctionnement du programme. Pour que le GPS externe fonctionne, il faut mettre à "1" la ligne `GPS_ENN`. Vous pouvez soit la mettre à "1" comme valeur par défaut dans le fichier `.ioc`, soit utiliser la fonction `HAL_GPIO_WritePin()`.



Testez le GPS externe en vous rapprochant de la fenêtre. On peut aller jusqu'à faire "pendouiller" le module à l'extérieur. Si cela ne fonctionne pas, passer à l'étape suivante.

6.10. Recherche de la ville la plus proche

Nous allons maintenant chercher la ville la plus proche et afficher son nom. Nous allons pour cela utiliser une "base de donnée" reprenant des structures nommées VILLE, ayant les données suivantes :

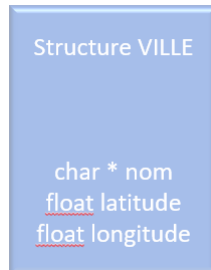


Figure 6.4.: la structure de donnée VILLE.

6.10.1. Création de la base de donnée

Ville	Latitude	Longitude
Kiev	50.4501	30.5234
Cergy	49.033	2.066
Chicago	41.88	-87.623
Barcelone	41.385	2.173



Définissez la structure VILLE dans un fichier .h.
Définissez quatre variables globales de type VILLE correspondant au tableau ci-dessus.
Définissez un tableau global contenant ces quatres villes.
Pour ces quatre opérations, vous pouvez vous inspirer du code ci-dessous.

```
// Définition du type de structure POINT dans un fichier .h
```

```
typedef struct point{  
    float x;  
    float y;  
}POINT;
```

```
// Définition de p1, p2, p3 dans un fichier .c
```

```
const POINT p1={0,0};  
const POINT p2={5,0};  
const POINT p3={0,5};
```

```
// p1,p2 et p3 ne sont pas modifiables.
```

```
// Définition de triangle dans un fichier .c
```

```
POINT triangle[3]={p1,p2,p3};
```

Listing 16: Inspirez vous librement de ce code définissant un triangle en tant que tableau de trois POINT.

6.10.2. Calcul de distance

Le calcul de distance entre deux points sur terre se fait ainsi : si l'on nomme Θ la latitude et Φ la longitude, on utilisera la norme suivante entre deux points.

$$norme = (\Theta_2 - \Theta_1)^2 + \left((\Phi_2 - \Phi_1) \cos \left(\frac{\Theta_2 + \Theta_1}{2} \right) \right)^2$$

Cette norme n'est pas la vraie distance, mais elle est suffisante pour déterminer le minimum de distance, qui est ce que nous cherchons. La vraie distance vaut $d = R \cdot \sqrt{norme}$ ou R est le rayon de la Terre.



Créez la fonction `float norme (VILLE * v, float latitude, float longitude);`

6.10.3. Recherche du minimum



En vous inspirant du TD3 (recherche de minimum) et de la question précédente, écrivez une fonction `VILLE * findNearest(VILLE tableau[], int size, float latitude, float longitude)`; qui renvoie un pointeur sur la VILLE la plus proche des coordonnées.

Testez votre programme en affichant la ville déterminée dans la console et sur l'écran OLED.

Pour utiliser l'écran, il y a trois étapes une fois le composant initialisé.

- Ecrire le texte ASCII en mémoire à l'aide de la fonction `snprintf(buffer,16,"Ville : %s",ville->nom);`
- Convertir le texte en mémoire en pixel "On/ Off" via la fonction `ssd1315_Draw_String(xPos,yPos,buffer,&Font_7x10);`
- Envoyer le tableau de pixel à l'aide de la fonction `ssd1315_Refresh();`

Pour en savoir plus sur l'usage de l'écran OLED, allez voir le tutoriel en ligne sur Moodle².

²Tutoriel réalisé par les stagiaires de première année 2024 : V. Montuclard et N. Haas

7. TP3 - Un framework multithreadé minimaliste.

7.1. Introduction - préparation

7.1.1. Programme en polling

Nous allons travailler sur le cahier des charges suivant :



On vous demande de faire un programme permettant de gérer les éléments suivants :

- Un chenillard fait clignoter 8 LEDs selon un motif.
- Un bouton poussoir permet de passer d'un motif à l'autre (chenillard vers le haut, vers le bas etc.)
- Deux boutons poussoirs permettent de modifier la vitesse du chenillard.
- L'écran OLED affiche le nom du motif ainsi que la vitesse du chenillard.

On dispose des drivers nécessaires, sous forme de fonctions :

- des fonctions `int ReadBTNSpeedUp(); int ReadBTNSpeedDown(); int ReadBTNMotif();`
- une fonction `void displayLED(int motif);`
- une fonction `void displayOnScreen(char * string);`

Ecrivez les grandes lignes du programme en mode polling.

Quel serait le problème de ce programme ?

7.1.2. Programme en multi tache simple

On va donc installer dans notre architecture un *scheduler*, un composant logiciel dont le rôle est de donner la parole à chacune des tâches les unes derrières les autres, comme indiqué ci-dessous. La conséquence de l'usage d'un scheduler est que chaque tâche doit s'exécuter rapidement. Cela interdit donc l'usage de fonction de type "HAL_Delay", ou bien "printf" (en effet, l'envoi sur une ligne de liaison série prend beaucoup de temps)

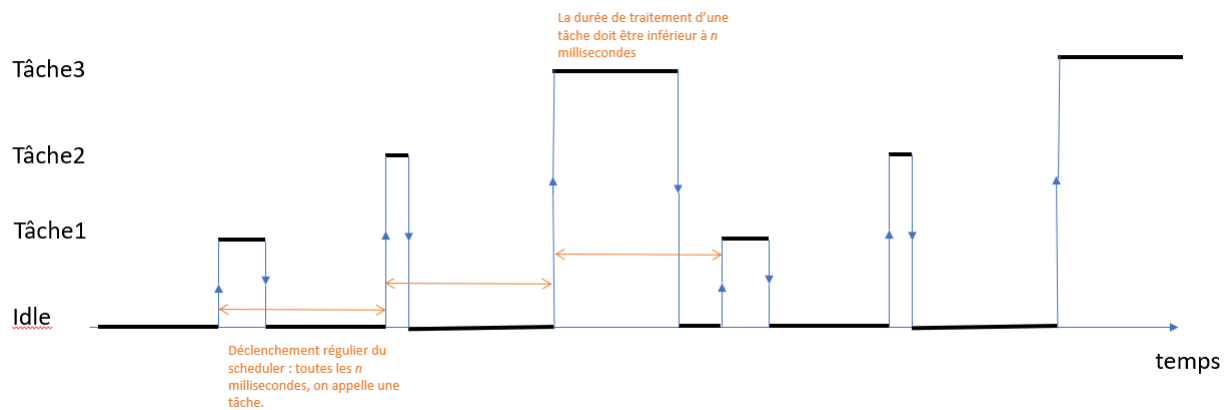


Figure 7.1.: Le fonctionnement du scheduler.

Le scheduler va donc utiliser un timer pour l'exécution régulière de tâche. Nous allons commencer par intégrer ce timer.

7.2. Mise en place du projet de base.



Téléchargez en ligne le fichier .ioc disponible dans l'espace moodle du cours décrivant l'état des périphériques pour ces manipulations.

Générez le code de base du projet. Attention, pour cela il faut être connecté au serveur "ST" (Menu myST).

Rajoutez le code permettant l'accès à la fonction `printf`, disponible en ligne sur Moodle (fonction `int __io_putchar(int ch);`).

Rajoutez un code `printf` dans le programme principal et testez le fonctionnement global du projet.

7.3. Le scheduler, usage du timer.

7.3.1. Calcul des paramètres du timer



En éditant le fichier .ioc du projet, relevez la fréquence d'horloge alimentant le timer 4.

Dans le même fichier, en regardant la valeur écrite dans les registre PSC et ARR du timer 4, quelle sera la période du scheduler ?

Compte tenu de la valeur précédente, et de ce que la liaisons série est à 115200 bits / secondes, quelle est la plus grande chaîne de caractères que l'on peut envoyer au cours d'une tâche.

Compte tenu de ce qu'il y a trois tâches, à quelle période s'exécutent-elles ?



- Créez deux fichiers task.c et task.h définissant trois tâches : void taskLED();, void taskButton(); et void taskScreen();
- Dans le corps du programme, avant la boucle infinie (car on ne veut effectuer ce traitement qu'une seule fois), démarrez les interruptions à l'aide de l'instruction HAL_TIM_Base_Start_IT(&htim4);
- Complétez la fonction HAL_TIM_PeriodElapsedCallback (déjà présente dans le main) selon le modèle ci-dessous pour l'appel des trois tâches. Ne modifiez pas le comportement du Timer 6.
- Testez à l'aide du débogueur, ou bien en affichant un message à chaque tâche.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    /* USER CODE BEGIN Callback 0 */
    static int taskNumber=0;
    if(htim->Instance == TIM4)
    {
        taskNumber=(taskNumber+1)%3;
        switch (taskNumber){
            case 0 :      taskLED();
                           break;

            // A vous de compléter ce code !
        }
    }
}
```

7.4. La tâche de gestion des LEDs.

7.4.1. Le driver logiciel des LEDs.

Les LEDs sont commandées à l'aide d'une structure de données :

```
typedef struct led {
    int isOn;
    GPIO_TypeDef* port;
    uint16_t pinNumber;
}LED;
```

Voici le code du driver de LEDs :

```

#include "led.h"
#include "main.h"

#define LED_BAR_SIZE 8

static LED led_bar[8]={0,LED_0_GPIO_Port,LED_0_Pin},{0,LED_1_GPIO_Port,LED_1_Pin},
    {0,LED_2_GPIO_Port,LED_2_Pin},{0,LED_3_GPIO_Port,LED_3_Pin},
    {0,LED_4_GPIO_Port,LED_4_Pin},{0,LED_5_GPIO_Port,LED_5_Pin},
    {0,LED_6_GPIO_Port,LED_6_Pin},{0,LED_7_GPIO_Port,LED_7_Pin}};

void LED_Set_Value_With_Int(int value){
    for (int i=0;i<LED_BAR_SIZE;i++){
        led_bar[i].isOn=(value>>i)&1;
    }
}

void LED_Set_Value_With_Array(int value[LED_BAR_SIZE]){
    for (int i=0;i<LED_BAR_SIZE;i++){
        led_bar[i].isOn=value[i]&1;
    }
}

int LED_Get_Value(){
    int toReturn=0;
    for (int i=0;i<LED_BAR_SIZE;i++){
        toReturn+=(led_bar[i].isOn)<<i;
    }
    return toReturn;
}

void LED_Update(){
    // A vous de compléter cette fonction
}

void LED_Test_All(){
    LED_Set_Value_With_Int(0xFF);
    LED_Update();
}

```



- Inclure ce code (led.c et led.h) disponible sous moodle dans votre projet.
- Complétez le code de la fonction void LED_Update(); qui scanne l'ensemble des éléments du tableau led_bar en les écrivant à l'extérieur via la fonction HAL_GPIO_WritePin(). L'objectif est de mettre à jour les 8 LEDs de la carte en fonction des valeurs du tableau "led_bar".
- Testez votre driver à l'aide de la fonction LED_Test_All();

7.4.2. La tâche taskLED().

Cette tâche utilise les déclarations suivantes :

```
static int globalDelayInMs=200;

typedef struct motif_type{
    int size;
    int * motif;
    char * name;
}MOTIF_TYPE;

int upDownMotif []={0,128,192,224,240,248,252,254,255,254,252,248,240,224,192,128};
int chenillardUpMotif []={1,2,4,8,16,32,64,128};
int chenillardDownMotif []={128,64,32,16,8,4,2,1};

const MOTIF_TYPE upDown={16,upDownMotif,"Up Down"};
const MOTIF_TYPE chenillardUp={8,chenillardUpMotif,"K 2000"};
const MOTIF_TYPE chenillardDown={8,chenillardDownMotif,"K -2000"};

const MOTIF_TYPE* tableau_motif[3]={&upDown, &chenillardUp, &chenillardDown};

static int index_tableau_motif=0;
```

Le type MOTIF est une structure contenant trois éléments :

- La taille du tableau.
- Un tableau de motifs les uns derrière les autres. Par exemple 1,2,4 etc... correspond à une led allumée à chaque fois.
- Le nom du motif.

La tâche taskLED commence par calculer le numéro du motif à écrire de la manière suivante :

```
int numero_motif=(HAL_GetTick())/globalDelayInMs%(tableau_motif[index_tableau_motif]->size);
```



- Comment évolue avec le temps la variable locale numero_motif ? Ce calcul permet, grâce à la connaissance de l'horloge, de nous affranchir de fonction de type HAL_Delay.
- Complétez la fonction taskLED() avec le calcul de numero_motif puis l'affichage du motif correspondant.
- Compilez avec différentes valeur de index_tableau_motif et testez. Cette variable globale sera modifiée lors des appuis boutons dans la suite de ce TP.
- Félicitations ! Votre chenillard est fonctionnel. Ajoutons maintenant les autres fonctionnalités.

7.5. La tâche de gestion des boutons.

Soit le code suivant :

```
typedef struct button {
    int isOn;
    int hasBeenPressed;
    GPIO_TypeDef* port;
    uint16_t pinNumber;
}BUTTON;

typedef enum direction{CENTER, DOWN, RIGHT, UP, LEFT}DIRECTION;

static BUTTON joystick[5]={0,0,BTN_CENTER_GPIO_Port,BTN_CENTER_Pin},
                        {0,0,BTN_BOTTOM_GPIO_Port,BTN_BOTTOM_Pin},
                        {0,0,BTN_RIGHT_GPIO_Port,BTN_RIGHT_Pin},
                        {0,0,BTN_TOP_GPIO_Port,BTN_TOP_Pin},
                        {0,0,BTN_LEFT_GPIO_Port,BTN_LEFT_Pin}
};

int BUTTON_Get_Value(DIRECTION direction){
    return joystick[direction].isOn;
}

int BUTTON_Get_Pressed(DIRECTION direction){
    return joystick[direction].hasBeenPressed;
}

void BUTTON_Update(){
    // A compléter par vos soins.
}
```



- Intégrez le code du driver de bouton, disponible sur Moodle à votre projet. (A vérifier : il est possible qu'il manque une ligne `#include "main.h"` dans `button.h`)
- Complétez la fonction `BUTTON_Update()`. Cette fonction vient lire la nouvelle valeur de chacun des boutons dans une variable locale. Puis, pour chaque bouton, la valeur `hasBeenPressed` vaut $(1 - \text{ancienne_valeur}) * \text{nouvelle_valeur}$. Enfin, on remet l'ancienne valeur à jour avec la variable locale.
- Quels sont les états possibles pour les éléments constitutifs de la structure `BUTTON` ?

7.5.1. La tâche `taskButton()`.

Dans cette tâche, on appelle `BUTTON_Update()`, puis on regarde les valeur de `BUTTON_Get_Pressed(UP)==1`, par exemple.



Codez le comportement attendu par le cahier des charges. Un printf à chaque appui de bouton permet de vérifier le bon fonctionnement...
Pensez à mettre à 1 une variable needDisplayRefresh pour la tâche taskScreen.

7.6. La tâche de gestion de l'écran.

On termine par la tâche de gestion de l'écran. Après avoir intégré les fichiers du driver d'écran disponible sur Moodle, complétez la tâche taskScreen(). Cette tâche ne change l'affichage si et seulement si nécessaire, et affiche les informations comme dans l'image ci-dessous.



Figure 7.2.: L'écran en fonctionnement.



Revoyez le tutoriel en ligne sur moodle sur l'écran pour le fonctionnement du système. Vous aurez besoin de la fonction `snprintf(char * buffer, int size, char * format, param...)`; (Pour mémoire la fonction `printf` écrit une chaîne formatée dans la console. `sprintf` écrit la même chaîne formatée dans la mémoire. `snprintf` écrit la même chaîne formatée dans la mémoire mais en ne dépassant pas la taille du buffer).

8. TP 4 - Des usages multiples des Timers

Lors du TP précédent, nous avons utilisé un timer pour sa fonctionnalité la plus élémentaire : la génération d'une base de temps. Nous allons maintenant utiliser ses fonctions de comptage plus avancés pour interfacer deux périphériques : un moteur pas à pas et un codeur rotatif.

8.1. Préparation : A FAIRE AVANT LE TP !

8.1.1. Environnement matériel - moteur

Le moteur de la carte est un moteur pas à pas intégrant un réducteur de vitesse. L'image ci-dessous, extraite de Wikipédia, indique le principe de ce type de moteur.

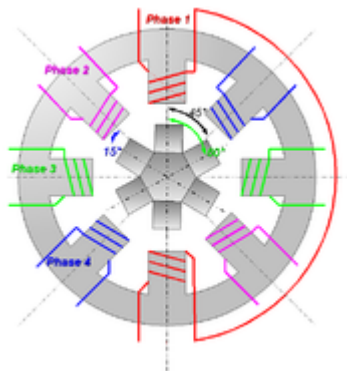


Figure 8.1.: Principe du moteur pas à pas (c) Wikipédia.

Le principe est le suivant : en alimentant la phase 1 avec une tension positive, on crée un champ magnétique sur lequel s'aligne les pôles les plus proches du rotor. Si on alimente ensuite la phase 2, le moteur va progresser de 45 degrés. On continue, et arrivé à l'alimentation de la phase 4, on passe à l'alimentation de la phase 1 à nouveau, mais cette fois-ci avec une tension négative. Au total on a donc 8 pas (4 phases * 2 alimentations possibles) dans ce moteur (et $8 \times 45^\circ = 360^\circ$, soit un tour complet). Un réducteur de vitesse augmente la résolution, c'est-à-dire le nombre de pas sur un tour de notre moteur.

Le moteur installé sur la carte possède 400 pas au total. On délègue à un composant spécifique, le TMC2225 le soin de générer les tensions positives et négatives sur le moteur. Ce composant est câblé à la carte par le biais des informations visibles sur la figure ci-dessous.

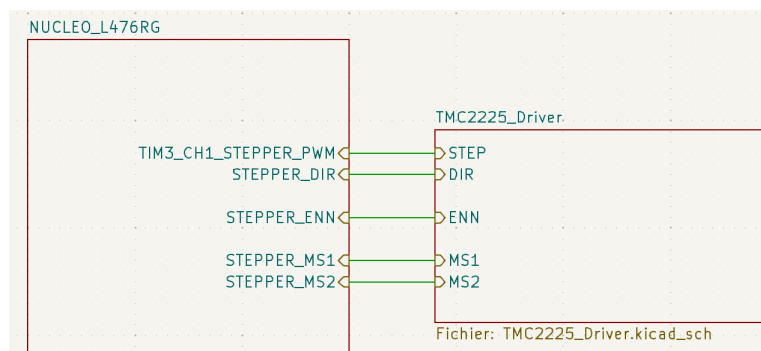


Figure 8.2.: Connection de la carte nucléo au TMC2225

Les connections "ENN", "DIR", "MS1" et "MS2" correspondent à des sorties GPIO du processeur, la connection "STEP" est une sortie PWM.

Nom symbolique	Fonction	Broche GPIO
ENN	Enable valide à l'état bas.	PA12
DIR	Sens de rotation horaire ou antihoraire	PA7
MS1, MS2	Micro - stepping	PA11, PB12
STEP	Avancé d'un pas	TIM3_CH1 (PA6)

Lorsque ENN vaut 1, le moteur est relâché, il n'y a pas de courant circulant dedans. Lorsqu'il vaut 0, on peut commander le moteur.

Le micro-stepping correspond au fait de pouvoir faire avancer notre moteur en micro-pas en alimentant simultanément deux phases.

STEP est une sortie de type PWM (cf le cours d'amphithéâtre) que l'on peut commander en vitesse (son rapport cyclique sera toujours de 50%).

8.1.2. Questions de préparation relatives au moteur pas à pas.



- Générez à l'aide de la méthode habituel un projet vierge pour ce TP.
- Relevez dans le fichier .ioc disponible en ligne la fréquence alimentant le Timer 3.
- Compte tenu de la valeur du registre PSC, quelle fréquence alimente le compteur du Timer 3¹ ?
- Si on souhaite une vitesse de 2 tours par seconde (120 tours par minutes, maximum atteignable dans notre configuration), quelle doit être la valeur du registre ARR ?
- Complétez le tableau ci dessous.

Vitesse	tr / min	tr / s	Valeur du registre ARR pour PSC=79
1	1	0.0167	
2	2	0.033	
3	5	0.083	
4	10	0.167	
5	30	0.5	
6	60	1	
7	90	1.5	
8	120	2	

8.1.3. Environnement matériel - le codeur rotatif

Un codeur rotatif est un système composé de deux interrupteurs, soit mécanique, soit optique, permettant de capter une rotation d'un pas, ainsi que le sens de rotation. Un codeur rotatif permet donc de capter un déplacement, mais pas une position absolue. Un bon exemple se trouve dans les automates de vente de ticket de RER à la gare de Cergy. On les met en œuvre, sous le nom d'odométrie, dès que l'on a besoin de mesurer un déplacement angulaire.

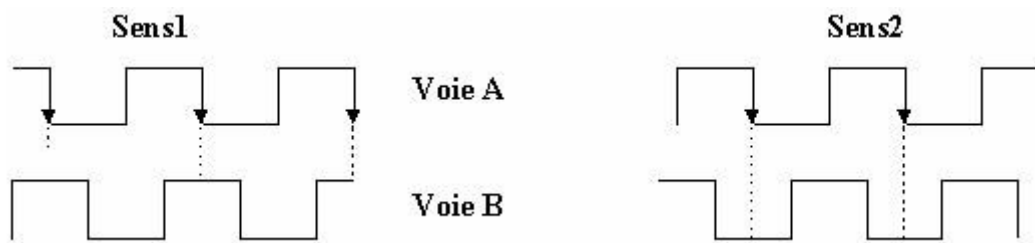


Figure 8.3.: Signaux de sortie d'un codeur incrémental en fonction du sens de rotation

Le codeur peut donc être directement relié aux canaux de capture d'un timer pour déterminer le déplacement par rapport à la position d'origine. C'est ce que l'on fait dans notre carte avec les signaux TIM2_CH1 et TIM2_CH2.

Le codeur incrémental se trouve sous l'écran OLED (ne pas le confondre avec le moteur) et a également un "click" de validation câblé sur PA15.

Vous trouverez dans l'extrait de schéma ci-dessous la connection encodeur-carte nucléo.

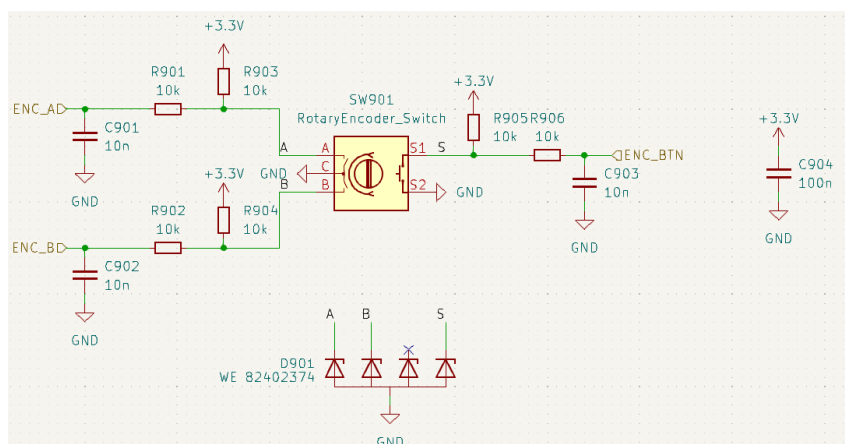


Figure 8.4.: Connection électrique de l'encodeur avec circuit anti-rebond.

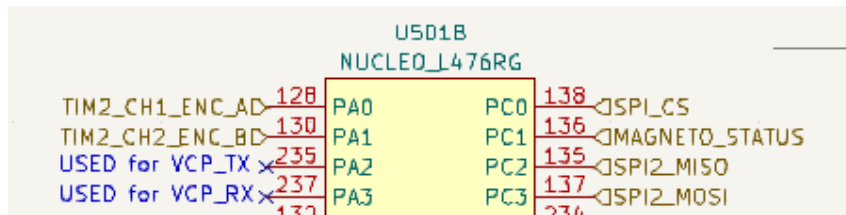


Figure 8.5.: Connection encodeur-carte nucléo.

8.2. Travail demandé - partie I : le driver du codeur incrémental



- Récupérez en ligne le fichier .ioc des TPs.
- Générez un nouveau projet intégrant les fonctionnalités habituelles (Ecran OLED, liaison série).
- Testez en affichant un message sur l'écran OLED et sur la console.
- Intégrez les déclarations ci-dessous aux endroits pertinents dans deux fichiers `encoder.h` et `encoder.c`.

```
typedef struct encoder{
    TIM_HandleTypeDef * htim;
    int32_t max_value;
    int32_t min_value;
}ENCODER;

void encoder_init(TIM_HandleTypeDef * htim_param, int32_t min, int32_t max);
int32_t encoder_read();

static ENCODER encoder;
```



- D'après le fichier .ioc, relever la configuration du Timer 2. D'après votre cours et la documentation, comment sont interprétés les signaux par le Timer en mode "encodeur".
- Codez la fonction `encoder_init`. Après avoir récupéré les paramètres dans la structure, elle fait appel à la fonction `HAL_TIM_Encoder_Start` dont vous chercherez la documentation.
- Dans la fonction `encoder_init` remettre le registre de comptage à 0 (`encoder.htim->Instance->CNT=0;`)
- Codez la fonction `encoder_read()` qui, dans un premier temps, renvoie juste le résultat du registre de comptage CNT. Pour cela, vous devez utiliser la syntaxe `encoder.htim->Instance->CNT`.
- Testez cette première version simple.
- Modifiez cette première version de manière à ce que si on dépasse la valeur maximum ou minimum, on renvoie la valeur saturée. La valeur saturée doit aussi être écrite dans le registre de comptage CNT.
- Testez en affichant la valeur saturée entre +8 et -8 sur l'écran OLED.
- Bonus : on ne rafraîchit l'écran si et seulement si la valeur mesurée est différente de la précédente (A ne faire que si vous avez le temps).

8.3. Travail demandé - partie II : le driver du moteur pas à pas



Cette partie est difficile, mais je pense que vous pouvez le faire. En cas de soucis, je met en annexe de ce texte de TP une solution pour que vous puissiez avancer. A vous d'utiliser cette ressource avec parcimonie en fonction de votre besoin.

- Récupérez sur Moodle le fichier `stepperMotor.h` donnant les structures ci-dessous.
- Ecrivez la fonction `init_stepper(...)` qui permet d'initialiser tous les éléments de la structure, puis positionne les bits de micro-stepping, et de direction. On démarre la PWM à l'aide de la fonction `HAL_TIM_PWM_Start()` dont vous chercherez la documentation. L'enable reste à 1 pour le moment.
- Ecrivez la fonction `launch_stepper()` qui sature la valeur de vitesse, écrit la direction en fonction du signe de la vitesse, puis enfin va chercher dans un tableau la bonne valeur de ARR selon la préparation faite plus haut.
- Ecrivez dans le registre CCR1 la moitié de la valeur de ARR pour assurer un rapport cyclique de 50 %
- Testez votre système, notamment avec une sonde d'oscilloscope permettant de valider vos calculs.

```
#include "main.h"
```

```
typedef struct stepperMotor{
    TIM_HandleTypeDef * htim_pwm;
    uint32_t pwm_channel_number;
    GPIO_TypeDef * gpio_direction;
    uint16_t gpio_direction_pin;
    GPIO_TypeDef * gpio_ms1;
    uint16_t gpio_ms1_pin;
    GPIO_TypeDef * gpio_ms2;
    uint16_t gpio_ms2_pin;
    GPIO_TypeDef * gpio_enable;
    uint16_t gpio_enable_pin;

    uint8_t speed;
    uint32_t step_target;
    uint32_t step_actual;
}STEPPERMOTOR;

void launch_stepper(int speed);
void init_stepper(      TIM_HandleTypeDef * htim_pwm_p, uint32_t pwm_channel_number_p,
                    GPIO_TypeDef * gpio_direction_p, uint16_t gpio_direction_pin_p,
                    GPIO_TypeDef * gpio_ms1_p, uint16_t gpio_ms1_pin_p,
                    GPIO_TypeDef * gpio_ms2_p, uint16_t gpio_ms2_pin_p,
                    GPIO_TypeDef * gpio_enable_p, uint16_t gpio_enable_pin_p);
```

8.4. Travail demandé - partie III : mise en lien des deux parties



Ecrivez un code permettant de choisir à l'aide du codeur incrémentale la vitesse de rotation du moteur. Le code affiche sur l'écran la vitesse du moteur.

8.5. Travail optionnel : amélioration du driver de moteur pas à pas.



En rajoutant un timer supplémentaire dont le rôle est de compter les impulsions de PWM, écrire une fonction qui permet de faire tourner le moteur à une valeur souhaitée pendant un nombre de pas fixé.

Le click central du codeur valide l'avancé en nombre de pas.

8.6. Annexe : une correction du driver

```
#include "stepperMotor.h"
#include "main.h"
#include "tim.h"
#include <stdlib.h>
```

```
static STEPPERMOTOR stepper_motor;
```

```

void init_stepper(          TIM_HandleTypeDef * htim_pwm_p, uint32_t pwm_channel_number_p,
                           GPIO_TypeDef * gpio_direction_p, uint16_t gpio_direction_pin_p,
                           GPIO_TypeDef * gpio_ms1_p, uint16_t gpio_ms1_pin_p,
                           GPIO_TypeDef * gpio_ms2_p, uint16_t gpio_ms2_pin_p,
                           GPIO_TypeDef * gpio_enable_p, uint16_t gpio_enable_pin_p)
{
    stepper_motor.htim_pwm=htim_pwm_p;
    stepper_motor.pwm_channel_number = pwm_channel_number_p;
    stepper_motor.gpio_direction = gpio_direction_p;
    stepper_motor.gpio_direction_pin = gpio_direction_pin_p;
    stepper_motor.gpio_ms1 = gpio_ms1_p;
    stepper_motor.gpio_ms1_pin = gpio_ms1_pin_p;
    stepper_motor.gpio_ms2 = gpio_ms2_p;
    stepper_motor.gpio_ms2_pin = gpio_ms2_pin_p;
    stepper_motor.gpio_enable = gpio_enable_p;
    stepper_motor.gpio_enable_pin = gpio_enable_pin_p;

    HAL_GPIO_WritePin(stepper_motor.gpio_direction, stepper_motor.gpio_direction_pin,0);
    HAL_GPIO_WritePin(stepper_motor.gpio_enable, stepper_motor.gpio_enable_pin,1);
    HAL_GPIO_WritePin(stepper_motor.gpio_ms1, stepper_motor.gpio_ms1_pin,0);
    HAL_GPIO_WritePin(stepper_motor.gpio_ms2, stepper_motor.gpio_ms2_pin,0);

    stepper_motor.htim_pwm->Instance->CCR1=stepper_motor.htim_pwm->Instance->ARR/2;
    HAL_TIM_PWM_Start(stepper_motor.htim_pwm,stepper_motor.pwm_channel_number);
}

void launch_stepper(int speed){
    int abs_speed_value = abs(speed);
    abs_speed_value = (abs_speed_value>8)?8:abs_speed_value;

    stepper_motor.speed=speed;

    const uint16_t period_counter_value[8]={1599,1399,1199,999,799,599,399,299};

    if (abs_speed_value==0){
        stepper_motor.htim_pwm->Instance->CCR1=0;
    }
    else{
        HAL_GPIO_WritePin(stepper_motor.gpio_enable, stepper_motor.gpio_enable_pin,0);
        HAL_GPIO_WritePin(stepper_motor.gpio_direction, stepper_motor.gpio_direction_pin,(speed>0)?1:0);
        stepper_motor.htim_pwm->Instance->ARR=period_counter_value[abs_speed_value-1];
        stepper_motor.htim_pwm->Instance->CCR1=stepper_motor.htim_pwm->Instance->ARR/2;
    }
}

float get_speed(){
    if (stepper_motor.speed==0) return 0;
    int32_t period_value = ((stepper_motor.htim_pwm->Instance->ARR+1)*80)/(stepper_motor.htim_pwm->Instance->ARR);
    int32_t one_turn = period_value * 1600;
    float speed = (((stepper_motor.speed>0)?1:-1)*(60 * 1000000))/one_turn;
}

```

```
    return speed;  
}
```

9. TP4 - Liaisons et bus, consommation électrique

Ce dernier TP donne lieu à deux manipulations : la mesure de courant consommé d'une part, et les liaisons et bus d'autre part.

9.1. Mesure de la consommation électrique.

9.1.1. Préparation

La mesure du courant se fait à l'aide d'une résistance de shunt entre load et supply.

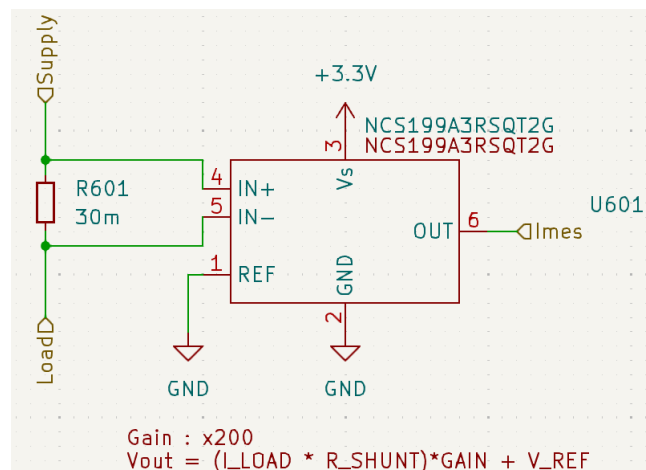


Figure 9.1.: Le câblage de la mesure de courant.

Load et Supply alimente exclusivement la carte Nucléo et les éléments passifs connectés (notamment les LEDs), selon le schéma ci-dessous. On ne mesure donc pas les consommations du moteur ou de l'écran et des autres éléments branchés sur le 3.3V.

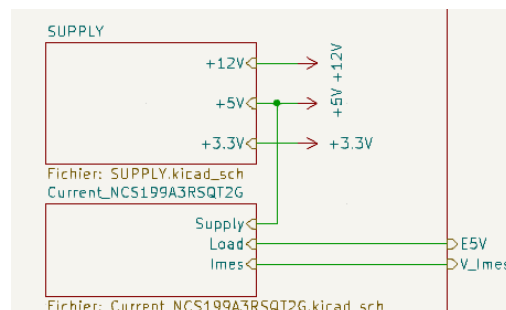


Figure 9.2.: Lien entre les alimentations LOAD et SUPPLY



Quelles sont les deux méthodes possibles pour mesurer un courant ? Quelles sont leurs avantages et inconvénient ?

La sortie de cet amplificateur mesure la consommation de quels éléments de la carte ?

Quel ordre de grandeur est attendu en courant ? Quelle est la chute de tension liée ?

Quelle valeur cela donne-t-il en tension en sortie de l'amplificateur différentiel ?

La grandeur I_{mes} est convertie en numérique au sein de l'ADC sur 12 bits. Quel est l'ordre de grandeur à lire ?

9.1.2. Manipulation de base



- Générez un projet vide à l'aide du fichier `.ioc`.
- Récupérez le programme en ligne sur moodle `mesure_consommation_courant.c` ainsi que `led.c` et les fichiers d'entêtes associés; appelez les méthodes `setup()` et `loop()` au sein du `main`.
- Placez une sonde d'oscilloscope sur la patte I_{mes} du connecteur J504.
- Dans la liaison série, relevez la tension utilisé leds allumées et leds éteintes (en appuyant sur l'interrupteur centrale du codeur rotatif).
- Commentez vos mesures.

Code à exécuter :

```

#include "main.h"
#include "mesure_consommation_courant.h"
#include "led.h"
#include <stdio.h>

void setup(){
    printf("**** TP5 : liaisons et bus, consommation electrique**** \r\n");
    HAL_TIM_Base_Start(&htim4);
    HAL_ADC_Start_IT(&hadc1);
}

void loop(){
    // HAL_PWR_EnterSLEEPMode(0,PWR_SLEEPENTRY_WFI);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    static int isOn=0;
    if(GPIO_Pin==ENC_BTN_Pin){
        printf("BTN pressed \r\n");
        LED_Set_Value_With_Int((isOn==0)?0:255);
        isOn=1-isOn;
        LED_Update();
    }
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc){
    if(hadc->Instance == ADC1){
        uint16_t adc_read=HAL_ADC_GetValue(&hadc1);
        float voltage = (adc_read*3.3)/4095;
        float intensity = (voltage*1000)/(0.4*200);
        printf("Mesure courant : %d \t %f \t %f\r\n",(int) adc_read, voltage, intensity);
    }
}

```

9.1.3. Mode power down



- Quelles sont les différences entre les modes suivants ?
 - HAL_PWR_EnterSLEEPMode();
 - HAL_PWR_EnterSTOPMode();
 - HAL_PWR_EnterSTANDBYMode();
- Intégrez le mode d'économie d'énergie le plus pertinent dans votre programme, et mesurez l'économie d'énergie réalisée.

9.2. Liaisons et bus

9.2.1. Le composant BMM150

Ce composant est fixé à l'extérieur de la carte-mère par le biais d'un connecteur JST.

Nous vous fournissons en ligne un driver relativement simple (sans structure, ouf !) à compléter. Nous allons nous concentrer sur les méthodes sur la méthode de lecture d'un registre.

Pour vérifier que le composant est correctement connecté à la carte, nous allons lire le numéro de série du composant. Celui ci est stocké dans le composant à l'adresse 0x40 et vaut forcément 0x32. Pour faire cette lecture il faut d'abord activer le composant en écrivant la valeur 0x1 à l'adresse 0x4b.

Les méthodes `initDriverBMM150`, `checkForBMM510Connection` et `readOneRegister` sont les suivantes :

```
static SPI_HandleTypeDef * localSPIHandler;
static uint8_t isConnected=0;

uint8_t initDriverBMM150(SPI_HandleTypeDef * param){
    localSPIHandler=param;
    checkForBMM150Connection();
    if (isConnected==1){
        writeOneRegister(CONTROL_REGISTER_ADRESS,CONTROL_REGISTER_NORMAL_MODE);
    }
    return isConnected;
}

uint8_t checkForBMM150Connection(){
    writeOneRegister(POWER_CONTROL_ADRESS,POWER_ON);
    uint8_t chip_id=readOneRegister(CHIP_ID_ADRESS);
    if (chip_id==CHIP_ID_VALUE){
        isConnected=1;
        printf("BMM150 connected \r\n");
        return 1;
    }
    printf("BMM150 connection failed");
    return -1;
}

uint8_t readOneRegister(uint8_t adress){
    uint8_t dataToSend=adress;
    // A modifier : le MSB de l'adresse doit forcément valoir 1.
    uint8_t receiveBuffer=0;

                                // Mise à 0 de la ligne de chip select
                                // Transmission de l'adresse modifié dans dataToSend
                                // Réception d'un seul octet
                                // Mise à 1 de la ligne de chip select.

    return receiveBuffer;
}
```



- Récupérez le code des fichiers bmm150.c et .h au sein de moodle et intégrez les à votre projet.
- Complétez la fonction `readOneRegister()`.
- Au sein de la fonction `setup()`, ajoutez un appel à la fonction `init-DriverBMM150(&hspi2)`
- Vérifiez le bon fonctionnement.
- Visualisez le transfert à l'aide d'un analyseur logique et explicitez-le.

9.2.2. Détection de l'angle par rapport au Nord magnétique

Pour notre dernière manipulation, nous allons compléter le driver...



- Complétez le code ci-dessous de la fonction "readManyRegister".
- Rajoutez une mesure régulière de l'angle.
- Affichez la soit dans la console (il faut alors enlever l'affichage de la consommation électrique) soit sur l'écran.

```
void readManyRegister(uint8_t address, int8_t * pData, uint8_t size){
    uint8_t dataToSend=address;
    // A modifier : le MSB de l'adresse doit forcément valoir 1.
    // Mise à 0 de la ligne de chip select
    // Transmission de l'adresse modifié dans dataToSend (1 seul octet)
    // Réception de size octets
    // Mise à 1 de la ligne de chip select.
}
```

9.3. Le projet complet (plus qu'optionnel)

Vous avez maintenant tous les éléments en main pour réaliser le cahier des charges suivant :

- Une boussole qui indique systématiquement le Nord. Pour cela, on fixe la carte BMM150 sur le moteur, on recherche le nord en minimisant la coordonnée X du capteur, ou bien en minimisant la fonction `readAngle()`.
- Tout en signalant la ville dans laquelle elle se trouve.
- Tout en indiquant sur un ensemble de LED la consommation électrique réelle du montage.



Amusez vous !

Il n'y a pas vingt manière d'apprendre l'informatique : cela se fait par la pratique. Il va donc vous falloir coder, tout le temps, des petits projets pour vous même ou pour l'école ou pour un employeur. Le C pour microcontrôleur vous permet en peu de ligne de code des réalisation importantes. Utilisez le !